AirOS 路侧感知系统 — 面试 QA 精简版（36 题）

> **这份是什么** 从 `0面试QA总表.md`（130 题）里挑出的**最核心 36 题**，用来考前突击、临场前快速过一遍。**题号沿用原表**（看到 Q2-3 就是原表的 Q2-3），想深挖某一题，回原表按题号查即可。 **不替代原表**——原表是字典，这份是常用字表。
>
> **挑选标准**：`[必背]` + `★★` 高频 + **诚实题**（容易穿帮的）+ 每个条目的**总览题**。 砍掉的是同类重复题和纯加分储备题——**不是说不考，是优先级低。**
>
> **怎么用**
>
> 1. **每个条目的第 1 题是"目录"**：答案里抛出的每个词，下面都有编号追问，按真实面试被问的顺序铺好了。
> 2. 其余题保留 1 条主追问，用来接住"为什么 / 会怎样"。
> 3. 答案都能**直接念出口**，念完 20~30 秒；念超时说明你加戏了。
>
> 🔴 **四条全局红线（全文通用，先看完再背）**
>
> 1. 全程说「**参与**」，不说「我从零设计」。
> 2. 视频那条说「**拉流和拆包**」，**绝不说解码**——本驱动没有解码环节。
> 3. 部署工具写「独立开发并维护升级」；**口语里主动说用了 AI 辅助**，别等被问出来。
> 4. 交通参数检测那条**另有专属红线**，写在该条开头，**先看完再背题。**
>
> ⏱️ **突击顺序建议**：开场 3 题 → 附录角色题 → 条目二 → 条目四 → 条目一 → 条目三。

目录

- 第 0 部分：开场自述（3 题）
- 条目一：设备抽象与插件化接入（7 题）
- 条目二：多路视频流接入（8 题）
- 条目三：设备与平台运维（5 题）
- 条目四：交通参数检测服务（9 题）
- 附录：通用追问（4 题）

第 0 部分：开场自述（3 题）

Q0.1 简单介绍一下你这个项目是干什么的？ [必背]

答： 我参与的是一个车路协同的**路侧系统**，跑在路口的边缘盒子上，干的事就是把相机、雷达、信号机这些设备接进来，数据收上来再发给平台和车端。C++ 写的，跑在容器里，硬件是带 GPU 的嵌入式板子。 我主要做了两块。**设备接入**这块：设备的**插件化接入**、**多路视频流**的拉取和链路调试、边缘设备的**运维**（GPU 指标采集，加上一个给现场用的批量部署工具）。**交通参数检测**这块：订阅上游融合感知输出的目标流，算路口的**流量、排队、车速、车头时距**这些指标，包括检测器框架、数据管道和车辆过线判定。

追问 1：边缘盒子是什么？为什么算力要放在路口，不上云？

> 就是放在路口机柜里的一台小工控机，带 GPU。必须在本地算是因为**延时和带宽**——视频流全量回传带宽扛不住，而车路协同要的是毫秒级响应，绕一圈云端就来不及了。云只用来做管理和长期存储。

追问 2：你说的"接进来"，具体接了哪几类设备？

> 主要是**相机、雷达、信号机**这几类。相机出视频流，雷达出目标点云，信号机给的是路口的灯态相位。它们协议完全不一样，所以才需要那套统一接入接口把它们抹平。

追问 3：为什么用 C++？又为什么跑在容器里？

> C++ 是因为**要压榨边缘设备的算力**，而且要在多线程下稳定跑几个月不重启。跑容器是为了**部署和环境隔离**——几十个路口的硬件批次不同，容器能保证我这份代码在任何一台上跑起来都一样。

追问 4：你们这个系统是分层设计的吗？

> 是。上面应用层、中间服务层、下面硬件抽象层。分层的好处是：加一个新设备不用动上面的代码，按标准实现一个"插件"挂上去就行。

追问 5：「插件化接入」具体是什么意思？

> 驱动编译成独立的动态库，框架启动时扫目录加载，驱动里一行注册宏把自己登记进工厂。所以加一款新设备是**加一个文件 + 改一行配置**，框架代码不用动。 👉 深挖见**条目一 Q1-1**。

追问 6：多路视频流是怎么组织的？

> **一路流 = 一份私有状态 = 一个独立线程**，一个设备管理多路。路跟路之间互不干扰，一路断了不影响别的路。 👉 深挖见**条目二 Q1-1**。

追问 7：运维具体运维什么？部署工具是干嘛的？

> 两块：一是**采 GPU 利用率、显存、温度**这些指标上报，看设备健不健康；二是我独立做的一个**桌面批量部署工具**，把"运维拿 Excel 一台一台 SSH 敲命令"变成"选表 + 点按钮"。 👉 深挖见**条目三 Q1-1 / Q3-1**。

追问 8：交通参数那块，你订阅的到底是什么数据？

> 订阅的是**上游融合感知输出的结构化目标流**——一帧里所有目标的清单，每条带 id、车型、经纬度、速度、车道号。**我从头到尾没见过像素**，解码和检测跟踪是算法团队做的。 👉 深挖见**条目四 Q1-1**。

追问 9：你这两块之间是什么关系？

> 是同一条业务链的**首尾两端**。前面我负责把设备数据接进来，中间解码、目标检测跟踪、多传感器融合是算法团队做的，他们产出结构化结果后我才在后面算交通参数。**中间那一棒不是我做的。**

> 📍 **这一答抛出了整张地图**——被追问时往哪儿走：
>
> - **"插件化接入"** → 条目一 Q1-1
> - **"多路视频流"** → 条目二 Q1-1
> - **"运维 / GPU 指标采集"** → 条目三 Q1-1
> - **"部署工具"** → 条目三 Q3-1
> - **"交通参数检测 / 订阅目标流"** → 条目四 Q1-1
> - **"两块什么关系 / 哪些不是你做的"** → Q0.3、附录 Q1

八股补给：

Q1：边缘计算和云计算，本质区别是什么？

> 就一条：**算力放在离数据多近的地方**。云是"数据送到远方的大机房算"，边缘是"在数据产生的现场就地算"。 选择边缘是因为两个硬约束：**带宽**——几十路视频全量回传，网络扛不住；**延时**——车路协同要毫秒级响应，数据绕一圈云端再回来就来不及了。 云并没有被取代，它负责的是**管理、配置和长期存储**这些不要求实时的事。

Q2：容器和虚拟机有什么区别？

> **虚拟机**是虚拟出一整台"电脑"——包括自己的操作系统内核，所以重、启动慢、占资源多。 **容器**是**共用宿主机的内核**，只把应用和它依赖的库**打包隔离**起来。所以它轻、启动快（秒级）、一台机器上能跑很多个。 代价是**隔离不如虚拟机彻底**（毕竟共用内核），以及**容器内看到的内核就是宿主机的**，所以跨硬件平台不像虚拟机那么自由。

Q3：为什么容器能保证"在哪台机器上跑都一样"？

> 因为它打包的不只是你的程序，还有**它依赖的所有库和系统工具**。同一份镜像在任何一台装了容器运行时的机器上，看到的"环境"是**一模一样的**。 这在我们的场景里特别关键——几十个路口的设备**硬件批次和出厂系统版本都不一样**，靠容器才保证我这份代码在任何一台上行为一致。 一句话：**容器解决的是"在我机器上明明是好的"这个千古难题。**

Q4：进程和线程有什么区别？

> **进程**是操作系统分配资源的最小单位——每个进程有自己独立的内存空间，互相看不见。**线程**是 CPU 调度的最小单位——一个进程里的多个线程**共享同一块内存**。 所以：线程之间传数据极快（直接读写同一份变量），代价是**必须自己管好锁**；进程之间互不可见，安全，但要通信就得走专门的方式（管道、共享内存、消息队列这些）。 我们"每路流一个线程"就是看中线程**共享数据方便**；而"驱动做成动态库"是**在同一个进程里**被加载进来的，不是独立进程。

> 背法：Q0.1 是**全表目录**。开场这 40 秒答完，面试官接下来问的每一句，基本都落在上面这张表里。

Q0.2 你说的「框架 + 组件」到底是什么意思？ [必背] ★ 高频

答： 框架是**插座**，组件是**插头**。框架只规定"插头必须长什么样"，不管你插的是台灯还是风扇。所以以后加新设备、加新算法，框架代码一个字都不用改。 具体到相机：框架负责扫描加载驱动、创建实例、把数据发出去；驱动负责按标准实现某款设备。两层中间靠一套**抽象接口**衔接。

追问：这个设计有什么不好？

> 两个。一是接口一旦定死，扩展就要动所有实现，所以定接口得先想清楚粒度；二是加载失败报错不直接，现象是"配置里写了这个名字但框架说找不到"，得翻日志定位。这也是我想改进的地方。

八股补给：

Q1：框架和库有什么区别？

> 一句话：**你调库，框架调你。** 库是你主动去调它——你在自己的 main 里决定什么时候调、调几次。库只是给你一堆工具。 框架是**它把流程搭好了，留出几个口子让你填**，你填的东西由框架在合适的时机去调。所以我们写驱动是"实现那几个约定的方法"，什么时候被调、调几次，是框架说了算。

Q2：那"面向接口编程"到底是什么意思？

> 就是**代码里只依赖"约定"，不依赖"具体是谁"。** 框架手里拿的是基类指针，它只知道"这个对象一定能喂帧、一定能出数"，至于背后是流量检测器还是车速检测器，它不关心、也不需要关心。 好处很直接：**换一个实现，框架不用改**；坏处是**多了一次间接跳转**（要查表才知道该调谁），以及**调试时看不到"到底调的是哪个"**。

Q3：接口定得太粗或太细，分别会怎样？

> **太粗**（一个方法塞一堆事）：所有实现类都得把不用的部分也写一遍，而且一改就要全部跟着改。 **太细**（拆成十几个方法）：调用方要连着调好几步才能办成一件事，**顺序错了就是 bug**，而且接口数量爆炸，谁都记不住。 判据是：**一个接口方法，应该刚好对应调用方的一次完整意图。** 我们那两个方法"喂一帧""出一次数"，就是按这个来的。

Q0.3 你做的这些，和「交通参数检测」是一回事吗？ [必背] ★★ 开场必被问

答： 两块都属于公司的路侧产品线，装在同一台边缘设备上，是同一条业务链的**首尾两端**：我前面负责把设备数据接进来、保证链路稳，后面负责订阅上游的感知结果算流量、排队这些指标。 中间那一棒**不是我做的**——

数据接进来要解码、做目标检测跟踪、多传感器融合，那是算法团队的活。他们给我的是**结构化结果**：一条条参与者记录，带类型、经纬度、速度、车道号。

> 🔴 **红线**：绝不能说"我拉的视频流直接给了流量模块"。中间隔着解码和感知融合，一追问就穿帮。

追问 1：那你拉的数据到底发给谁了？

> 发到框架的**消息总线**上了，谁订阅谁收到。具体谁是订阅方属于感知融合那一块，是算法团队的模块，我没参与。我只保证这一棒是稳的——能连上、断了能重连、出问题能从指标上看出来。

追问 2：那为什么两边用的通信方式不一样？

> 不同团队的选型。主框架用一套通信中间件加 protobuf；交通参数检测那边的形态是**算法插件**，跑在一个宿主程序里，宿主给它的是 C 接口加另一套通信库，数据格式是 JSON。两边业务上靠**标准化的感知消息**衔接，通信各走各的。

追问 3：所以到底哪些是你做的？

> 我做的：多路视频流的拉取、设备的插件化接入、GPU 指标采集、部署工具，还有交通参数那边的检测器框架、过线判定、生产消费管道。**没做的**：图像解码、目标检测和跟踪、多传感器融合。

八股补给：

Q1：把一个大系统切成好几个模块，边界该怎么划？

> 判据是**看变化的方向**。会**一起变**的东西放一个模块里；**各自变**的东西拆开。比如"加一款新相机"只应该动驱动那一个文件，如果它还要动框架，那边界就划错了。 另一个判据是**依赖方向要单向**：下游认识上游，上游不认识下游。如果出现"A 要用 B、B 也要用 A"，那基本就是设计出了问题，得抽一层出来打破这个环。

Q2：模块之间怎么传数据才算"解耦"？

> 最低要求是**不直接认识对方**：不 include 对方的头文件、不调对方的函数、不知道对方的存在。中间靠一个**双方都认的东西**衔接——可以是一份消息定义（protobuf / JSON）、一个接口、或者一个消息队列。好处是谁重写都不影响另一边；代价是**多一层转换**（要序列化、要约定格式），以及**调用栈断了，调试时要靠日志去串**。

Q3：同一个进程内的两个模块，和两台机器上的两个服务，通信方式有什么不同？

> 进程内最简单——**直接函数调用**，或者传个对象指针，最快。但两边就要互相依赖，耦合度高。 跨进程或者跨机器，就必须走**通信中间件**：共享内存、消息队列、socket、或者 DDS 这类。它们都能做到"两边不认识"，代价是**多一次序列化 + 一次传输**。 所以选型的问题其实是：**你更在乎快，还是更在乎两边能独立演进。**

📌 *自己复习：* `6两个模块-合成一个故事.md` + 图 `imgs/07-两个模块怎么串成一条链路.svg`

条目一：设备抽象与插件化接入（7 题）

简历原话 设备抽象与插件化接入：为相机等路侧设备定义统一抽象基类接口（初始化、启停、数据下发、状态查询），各厂商驱动通过虚函数重写实现同一套接口；配合工厂模式 + 注册宏完成驱动自注册与动态库加载，新增设备型号只需新增驱动文件、框架代码零改动，接入方式从"改代码"变为"改配置"。

🔴 **本条两条红线**

1. 全程说「**参与**」——接口和工厂是 mentor 定的架构，我负责具体模块落地。
2. "框架代码零改动"指的是**新增一款设备**不用改框架；**新增一类能力还是要动接口**，被追问要能说清这个边界。

❗ **"统一"是按设备类型各一套**（相机 / 雷达 / RSU / 信号灯……各一份接口 + 工厂），**没有所有设备共用的公共基类。** ❗ **别把"厂商"说成"厂商 SDK"**：我做的是**通用 RTSP + FFmpeg** 协议层接入，**没对接任何厂商私有 SDK。**

技术点 1：统一接口与多态（1 题）

Q1-1 你说的「统一接入接口」，具体是怎么设计的? [必背]

答： 就三步。先写一个**抽象基类**，把四个**纯虚函数**列上去——初始化、启动采集、查状态、下发指令；再让每家厂商的驱动**继承**这个基类、把四个函数各自实现出来；最后框架手里只拿**基类指针**，只管调，不管跑的是哪一份实现。 另外留了一条**回调**给数据上行：设备采到数据就丢给它，至于交给谁由框架定，驱动自己不知道。 能统一成这四个函数，是因为不管哪家设备，一辈子干的事都一样——先配资源、再出数据、中间要能查健康、有的还得能反向下发指令。差别只在每一步具体怎么做。



橙色回环 = 数据上行：驱动只负责把数据交给回调，不关心发给谁

> 看图要点：从上往下是「框架 → 工厂 → 抽象基类 → 具体驱动」，橙色回环是数据上行。上面三层只认**抽象基类**这一个类型，最底下那个驱动是后加的，框架一行都不用改。

> ❗ **别只答"虚函数基类 + 子类重写"**——那只讲了多态，没讲插件化。完整的是**三件套**：
>
> 1. **抽象基类**（纯虚函数）——定能力清单；
> 2. **工厂 + 注册宏**——驱动文件末尾一行注册宏，把"名字 → 构造函数"登记进表，**发生在 main 之前**；
> 3. **动态库加载**——驱动编译成独立 `.so`，启动时扫目录加载。
>
> ❗ **"统一"是按设备类型各一套**（相机、雷达、RSU、信号灯……各一份接口 + 工厂），**没有所有设备共用的公共基类**。被追问就诚实说：按类型分别统一，抽公共基类是后续优化点。
>
> ❗ **别把"厂商"说成"厂商 SDK"**：我做的是**通用 RTSP + FFmpeg** 的协议层接入，一套代码通吃各家，**没对接任何厂商私有 SDK**。

追问 1：那这个接口在代码里是怎么落地的?

> 就是一个**抽象基类**，四个方法全是**纯虚函数**——基类只声明不实现，谁继承谁就必须自己写。框架手里拿的是基类指针，调的时候自动跑到具体那份实现上，这就是运行时多态。

追问 2：为什么正好是这四个方法？不能多不能少?

> 因为不管哪家设备，**一生的动作都一样**：先配置资源、再开始出数据、中间要能查健康、有的还要能反向下发指令。这四个覆盖了设备全生命周期，差别只在每一步怎么做。 ⚠️ 真遇到某家不支持其中某个，让它**空实现返回不支持**就行，不能为了迁就它去改接口——改接口会波及所有已接入的驱动。

追问 3：为什么用纯虚函数？普通虚函数不行吗？

> 不行。普通虚函数意味着基类**给了默认实现**，子类不写也能编译过——运行时就变成一个"什么都没干"的静默失败，现场查起来极其痛苦。纯虚函数把这件事**逼到编译期**：漏实现就编译不过。 而且基类本来也给不出有意义的默认实现——一台相机怎么初始化，基类怎么会知道。

追问 4：用基类指针销毁对象，要注意什么？

> **基类析构函数必须是虚函数。** 否则通过基类指针 delete 的时候，只会调到基类的析构，子类那部分资源就泄漏了——而且这种泄漏是静默的，跑几天才看出来。 这也是我判断一个"面向接口"的设计成不成熟时最先看的一点：**只要打算用基类指针持有对象，析构就必须虚。**

追问 5：那条回调为什么选择在构造函数里传进来？

> 这叫**构造期注入**。最大的好处是：回调变成对象一旦存在就**必然可用**，不会出现"建好了但还没设回调"这种半初始化状态。如果做成 setter，就多了一个必须记得调用的步骤，忘了就静默丢数据。

追问 6：这套接口最大的问题是什么？重做你会怎么改？

> 两个。一是**接口一旦定死，扩展就要动所有实现**，所以定接口得先想清楚粒度；二是**各类设备各有一套接口，没有公共基类**，同构代码重复了九份。重做的话我会先抽一层真正公共的基类，再往下按类型分化。

✅ **一句话版本（面试照背）**

它的核心确实是抽象基类和虚函数重写——接口层把能力定成纯虚函数，驱动继承下来各自实现，框架只拿基类指针调。但光有这层还不叫插件化：我在接口之外还配了**注册工厂 + 动态库加载**，驱动里一行注册宏把自己登记进工厂，框架侧只认配置里的一个名字。所以新增一个型号，是加一个驱动文件 + 一行注册，框架代码一行都不动——这才是我说的「统一接入」。

八股补给（面试官爱往底层挖的，答不上很伤）：

Q1：什么是多态？有哪几种？

同一个调用、跑出不同结果。分两种：**编译期多态**——函数重载、模板，编译时就定死了调哪个；**运行期多态**——虚函数 + 基类指针，运行时才决定调哪份实现。我这块用的是后者。

Q2：只有纯虚函数的类，能 new 出来吗？

不能，这种叫**抽象类**（接口类），只要有一个纯虚函数没实现就不能实例化。它的用途就是当"规格说明书"给别人继承的。

Q3：那这个类里能写普通成员吗？

能。可以有普通成员函数、也可以有成员变量（比如我们这个基类里就有一个回调成员）。"全是纯虚函数"只是我们这个场景的选择，不是抽象类的要求。

📌 **自己复习（面试别说）：**

- 接口：`base/device_connect/ipcamera/device_base.h` 的 `IpCameraDevice`，四个纯虚 `Init / Start / GetState / WriteToDevice`，`protected: IpCameraCallBack sender_`，回调由构造函数注入。
- 子类：`standard_ipcamera.h:44` `StandardIpCamera : public IpCameraDevice`，四方法全 override。
- 工厂：`device_factory.h` 单例 + `map<string, CONSTRUCT>`；宏 `V2XOS_IPCAMERA_REG_FACTORY(StandardIpCamera, "standard_ipcamera")`，注册点 `standard_ipcamera.cc:608`。
- 框架取实例：`ipcamera_component.cc:42` `GetUnique(conf_.device(), ...)`；同文件 :31 动态加载 `device/lib/ipcamera/`。
- 同构共 9 套（camera / ipcamera / lidar / radar / rsu / traffic_light / mec / ins / cloud），每类一份 device_base.h + device_factory.h。
- 老的 `modules/camera/ipcamera/include/camera.h`（按 `Vendor` 分海康/大华/仿真）走厂商 SDK 路线，跟 standard_ipcamera 不是一条路，别混讲。

技术点 2：工厂 + 自注册（3 题）

Q2-1 为什么要用工厂模式？直接在框架里判断类型创建对象不行吗？ [必背] ★ 高频

答： 好处就一句话：**加一款新相机，框架不用动。** 不用工厂的话，框架里得写一串 **if-else**：配置里写 A 牌就 new A 牌那个类、写 B 牌就 new B 牌的——每来一款新相机都要打开框架加一个分支、整个重编一遍，而且框架得把各家驱动的**头文件**都 include 进来。 用了工厂，框架手里只有一张 **map**：**key** 是配置里写的设备名，**value** 是"这个对象怎么造"。框架报个名字，工厂在 map 里一查、new 出来交给它，框架压根不认识背后是哪

个**类**。 那这张 map 是谁填的？**驱动自己填的**——每个驱动文件末尾写一行**宏**（`#define` 那套），宏展开后是个静态对象，动态库一加载它就自动把自己的名字和造法塞进 map。所以新相机就是加一个 .cpp 文件，框架一个字不改。 打个比方：map 就像前台的取件码本子——你报码、他查本子、把包裹递给你，你不用知道仓库在哪。

追问：那这不就是把 if-else 挪到工厂里了吗?

> 区别在于**那张 map 是谁填的**。挪进工厂的话，工厂还是得认识所有牌子，加一款照样要改工厂。我们这个是**驱动自己往 map 里塞**——新相机写一个 .cpp、末尾一行宏，工厂完全不知道世界上有哪些设备。所以代价从"改工厂"变成了"加个文件"。

八股补给（这块面试官爱往底层挖，答不上很伤）：

Q1：什么是工厂模式?

> 就是把"new 对象"这件事交给专门一个地方，调用方只说要什么、不自己 new。三种常见写法：**简单工厂**（一个类里写 if-else）、**工厂方法**（每个产品配一个工厂类）、**抽象工厂**（造一整族产品）。我这个是简单工厂 + 一张 map，再加个自注册。

Q2：为什么用 map 存，不直接写 if-else?

> 为了**好扩展**。if-else 是写死的，加一种就要改一次代码、重编一次；map 是运行时查表，加一种只是往表里多塞一条。查找上 map 是红黑树、O(log n)，几十种设备根本不是瓶颈。

Q3：你这个工厂为什么是单例？单例怎么写线程安全?

> 全进程只需要**一张** map，做成单例才能保证大家查的是同一张表。线程安全靠写法：C++11 之后，函数里的**局部静态变量**编译器会加锁，用到才构造，天然安全。老写法（new 个指针 + 双重检查锁定）特别容易写错。

Q4：宏（#define）有什么坑?

> 三个：**没类型检查**、**不好调试**（报错报的是展开后的代码）、**容易命名冲突**（所以我们宏里要用 `##` 把类名拼进变量名，不然多个驱动会重复定义）。这是我觉得这套设计里最该被换掉的部分。

Q5：那单例有什么缺点?

> 本质是个全局变量——测试的时候不好替换，依赖关系也被藏起来了。不是什么东西都该做成单例，只有**确实全局唯一的**才值得。

> 📌 *自己复习（面试别说）：工厂是 `device_factory.h` 单例 + `map<string, 构造方式>`；抽象基类 `IpCameraDevice` 四个纯虚方法；宏 `V2XOS_IPCAMERA_REG_FACTORY(...)` 注册点 `standard_ipcamera.cc:608`；框架侧 `GetUnique(conf_.device(), ...)` 只按配置名取，返回基类指针。其余术语（运行时多态 / 编译期依赖 / 自注册 / 静态对象 / 动态库加载）心里有数就行，口播不抛。*

Q2-3 你说的「一行注册宏」，它到底是怎么工作的？ [必背] ★★ 高频

答： 每个驱动文件末尾写一行宏。这行宏展开之后，其实是定义了一个**静态对象**，它的**构造函数**干的事就是：把自己的设备名和"怎么造自己"塞进工厂那张 map。 关键就在"静态"两个字——C++ 保证静态对象的构造函数会在程序加载时**自动执行**，不需要谁手动去调。所以框架把驱动加载进来，注册就自动完成了，不需要在任何地方维护一份"设备清单"。

追问：但是静态对象的初始化顺序是不确定的，会不会出问题？

> 这个担心是对的，C++ 确实不保证不同 `.cpp` 之间静态对象的初始化顺序。但这里恰好安全，关键在于**工厂是用函数内的局部静态变量实现的单例**——用到的时候才构造，所以不存在"注册跑到一半、工厂还没建好"。 反过来如果工厂是个全局变量，注册可能在它构造之前发生，往一个还没初始化的 map 里塞东西就是**未定义行为**。

八股补给：

Q1：宏展开之后大概长什么样？

> 大致是 `static 某某注册器 g_某某注册器("设备名");`——一个静态对象，构造函数里干正事。那个变量名靠 `##` 把类名拼出来，保证每个驱动不重名。

Q2：为什么非得用宏？写个普通函数不行吗？

> 宏的好处是**一行搞定、不容易写漏**。但代价很实在：**没类型检查、报错信息是展开后的代码、调试器里看不到原样**。所以宏不算好设计，只是当时"让驱动作者少写几行"这个收益更大。要改我会换成显式注册函数。

Q3：你说的"静态对象"和"静态成员变量"是一回事吗？

> 不是。这里是**文件作用域的静态对象**（全局对象加了 `static`，作用域只限本 `.cpp`）；静态成员变量是类里的、属于类而不属于某个对象。两个概念别混着讲。

追问 1：静态对象到底什么时候构造？动态库呢？

> 可执行文件里的静态对象，在 **main 之前**就构造好了；动态库里的是**加载时才构造**——也就是框架扫描目录、把 `.so` 加载进来的那一刻。所以注册这个动作发生在框架真正开始工作之前，等它去查表时表里已经有东西了。

追问 2：宏里面要用符号拼接生成变量名，为什么？

> 因为宏在**每个驱动文件里都要用一次**，如果不拼接就会生成同名的全局变量，链接期直接报重复定义。拼接上类名之后，每个驱动拿到的是自己那份独一无二的变量名。

追问 3：工厂是单例，多线程同时调用安全吗？

> 要分两段看。**注册**发生在加载期，那时候还没起工作线程，天然串行；**查询创建**在运行期，多线程可能同时进来。所以查表那一步要加锁保护——这也是为什么我倾向于让注册越早完成越好，运行期只做只读查询。

追问 4：如果两个驱动注册了同一个名字会怎样？

> 后加载的那个会把先来的**覆盖掉**，运行时的现象是"配了这个名字但行为不对"，而且不报错、很难查。所以更稳妥的做法是**重复注册时直接报错退出**，而不是静默覆盖。

追问 5：宏用起来爽，代价是什么？

> 调试很痛苦——宏展开后的代码和看到的不一样，报错信息也难读。而且它是**编译期**的黑魔法，出了问题只能看预处理后的文件排查。这是我认为这套设计里最该被替换掉的部分，更好的做法是改成显式注册函数。

Q2-8 用具体流程说说：新来一款设备，你要做哪些事？ [必背] ★★ 高频场景题

答： 四步，**全程不碰框架代码。** 第一，新建一个 `.cpp`，写一个类**继承**那套统一接口，把那几个纯虚函数实现出来； 第二，文件末尾加一行注册宏，名字填"厂商 + 型号"； 第三，把这个模块编译成独立的 `.so`，丢到框架会去扫描的那个目录下； 第四，改配置，把里面的设备名从老驱动改成新的。 重启之后框架扫描目录、加载 `.so`、注册自动完成，就按新配置创建实例了。**框架代码一行不改，也不用重新编译框架本身。**

追问：这套流程的瓶颈在哪？

> 两个。一是**接口是有限的**，遇到厂商有特殊能力就得回头扩展接口；二是**出问题只能靠日志排查**，没有"已注册清单"的时候比较绕。这两个都是我前面说的改进点。

八股补给：

Q1：编译成 .so 要加哪些选项？

> 两个都要：`-fPIC` 和 `-shared`。`-shared` 告诉链接器产出的是动态库而不是可执行文件；`-fPIC` 生成**位置无关代码**。64 位下少了 `-fPIC` 基本链接不过。

Q2：为什么动态库必须用 -fPIC？

> 因为同一个 `.so` 会被加载到**不同进程的不同地址**，代码里不能写死绝对地址。`-fPIC` 让所有跳转和取址都走**相对偏移**或者 GOT 表，这样放哪儿都能跑。

Q3：静态库（.a）和动态库（.so）怎么选？

> 静态库在**链接期**就被整个拷进可执行文件——程序变大，但部署简单、不依赖外部环境；动态库在**运行期**才加载，多个程序能共用一份，更新也不用重编主程序。我们选动态库，就是为了"加设备不用动框架"这一点。

技术点 3：动态加载（2 题）

Q3-2 加载失败会有什么现象？你怎么排查？ [加分] ★ 有实战感

答： 现象是**设备名查不到**——配置里明明写了这个名字，但创建实例的时候失败，打一条错误日志。 麻烦的是失败原因不会直接暴露，要用 `dlerror()` 才能拿到人话的错误信息。我一般三步排查：用 `ldd` 看这个库是不是缺了某个依赖；确认那个依赖在搜索路径里能找到；把依赖补上，或者在加载目标库之前**先显式地把依赖加载进来**。

追问：你实际遇到过吗？

> 遇到过。有次一个 `.so` 始终加载失败，现象就是它的设备注册不上。后来发现它依赖了某个第三方库，而那个库当时还没被加载。解决办法是加载驱动之前先检查依赖在不在，不在就按顺序先加载。这类问题**编译期完全发现不了**，只能靠启动日志。

八股补给：

Q1：`ldd` 是干什么的？

> 列出这个 `.so`（或可执行程序）依赖哪些库、以及每个库**实际解析到了哪个路径**。输出里出现 `not found` 就是缺依赖，这是最常见的一类问题。

Q2：找不到库的时候，系统会去哪儿找？

> 大致顺序：可执行文件或库里写死的 `RPATH` / `RUNPATH` → 环境变量 `LD_LIBRARY_PATH` → `/etc/ld.so.cache`（由 `ldconfig` 生成）→ 最后才是 `/lib`、`/usr/lib` 这些默认路径。所以现场要么改环境变量，要么把库放到标准路径后跑一次 `ldconfig`。

Q3：为什么这类问题编译期发现不了？

> 因为编译期只检查"你代码里调的函数有没有声明"，而动态库加载是**运行期**的动作——库在不在、依赖全不全，只有真正加载那一刻才知道。这正是"灵活"换来的代价。

Q3-6 动态库到底是怎么被加载进进程的？底层做了什么？ [必背] ★★ 八股必考

答： 就三步：**打开库 → 找到要用的东西 → 用完关掉**，对应 `dlopen`、`dlsym`、`dlclose` 这三个接口，出错的原因靠 `dlerror` 拿。 具体说：框架扫到某个驱动目录，读出目录里那个描述文件（里面写着这个驱动的动态库叫什么名字），拼出 `.so` 的完整路径，然后调加载接口**把这个文件映射进进程的地址空间。 就在这一刻**，库里那个全局静态对象被构造、注册宏被执行，驱动把自己登记进工厂——所以"**加载**"和"**注册**"是**同一步**发生的，不是两件事。 加载时传的标志位是 `RTLD_NOW`，意思是**立刻把库里所有还没解析的符号全部解析掉**，缺一个就整

个失败。相对的那个是"用到才解析"，加载快但可能跑一半才炸。**我们选当场解析——宁可启动就失败，也别跑到一半崩。**

追问：那你这个项目是直接用 `dlopen` 吗？

> 不是，用的是它的升级版，多一个参数可以指定**加载到一个独立的命名空间**里。好处是两个库即使有同名符号也互不干扰——这对"各家厂商的库要塞进同一个进程"的场景挺关键。 还有个细节：真正加载之前，会先用"**只查询、不加载**"的标志**试探一次**，看这个库是不是已经在进程里了；已经在就直接跳过，避免重复加载。

八股补给：

Q1：加载动态库一共要用到哪几个接口？

> 四个，都声明在头文件 `dlfcn.h` 里： `dlopen`——打开并加载一个 `.so`，返回一个句柄（一个 `void*`），代表"这个库在这个进程里的一份实例"。 `dlsym`——**按名字在库里查一个符号的地址**，函数或者全局变量都行。 `dlclose`——关掉句柄，引用计数减一。 `dlerror`——拿上一次失败的人话错误信息，没有错误就返回空。 还有个容易忘的：**编译的时候要链 `-ldl`**，不然会报这几个接口找不到。

Q2：`RTLD_NOW` 和 `RTLD_LAZY` 有什么区别？

> 它俩是 `dlopen` 的第二个参数，决定**什么时候去解析库里那些还没找到的符号。** `RTLD_NOW`：加载时就**全部解析完**，有一个找不到就整个加载失败。启动慢一点点，但**问题在启动那一刻就暴露。** `RTLD_LAZY`：**用到哪个才解析哪个**，加载快，但可能程序跑了一半、调用到某个没解析的函数时才炸。 长期跑的服务基本都选前者——**早点失败，比半夜崩好。**

Q3：`dlsym` 是干什么的？我们这个项目里用到了吗？

> 它是"**按名字在库里找东西**"的接口。编译的时候你根本不知道这个库将来会被加载到哪个地址，所以只能运行期拿着名字去查，查回来一个函数指针再调用。 **设备驱动这块没用到它**——驱动是靠库里那个全局静态对象**自己报到工厂**的，框架不需要知道库里有什么函数。 而**算法插件那块用到了**：那边是显式地按名字去取一个"创建对象的函数"，取到之后才能造对象。两种路子的区别一句话：**一个是库主动报到工厂，一个是框架主动去库里找。**

Q4：动态库加载进来之后，代码和数据是共享的还是每个进程一份？

> **代码段是共享的**：同一个 `.so` 被十个进程加载，物理内存里**只有一份代码**，靠内存映射挂到各自的地址空间——这是动态库省内存的主要原因。 **数据段是每个进程各一份**：库里的全局变量、静态变量，每个进程都有自己的副本，改了互不影响。 所以**别想靠动态库的全局变量做进程间通信**，那玩意儿根本不共享。

Q5：用 `dlerror` 拿错误信息有什么要注意的？

> 它返回的是**最近一次失败的**原因，而且**取一次就清空了。**所以必须**紧跟在失败的那个调用后面立刻取**，中间不能再插入别的动态加载调用，否则拿到的就是别的错误、甚至是空。 另外它返回空**不等于一定成功**，只代表"这次没有新错误"。判断成败还是要以 `dlopen` 的返回值是不是空为准。

技术点 4：改配置不改代码（1 题）

Q4-1 你说的「改配置」，具体改的是哪几层? [必背] ★

答： 一共三层，分工很明确。 第一层是**进程编排文件**：决定这个进程要加载哪个 `.so`、实例化哪个类、配置文件在哪； 第二层是**组件配置**：里面最关键的字段就是**用哪个驱动**，以及指向下一层配置文件的路径； 第三层是**驱动自己的业务参数**：比如这台设备要连哪几路视频流地址、每路输出到哪个通道、多久重连一次。 一句话：**第一层决定跑什么进程，第二层决定用哪个驱动，第三层决定这台设备具体怎么连。**

八股补给：

Q1：这些配置文件一般用什么格式?

> 常见三种：`JSON`（人好读、各语言都有库，我们交通参数那块用的就是它）、`protobuf`（和代码里的结构体强绑定，改字段编译期就能发现问题）、`YAML`（手写最省心，适合人经常改的场景）。

Q2：配置读进来之后放哪?

> 一般是解析到结构体里、由一个**全局单例**持有，**启动时读一次**，之后运行期不再读文件。我这个项目就是这样——只在 `init` 的时候加载，检测器构造时把需要的几何信息**值拷贝**进自己的成员。

Q3：为什么只在启动时读一次?

> 两个原因：一是**没必要**，每次用都去读文件纯浪费；二是**避免状态不一致**——读到一半配置变了，前面的模块用老值、后面的用新值，这种 bug 极难查。真要热更新就得整体切换，不能改一半。

条目二：多路视频流接入（8 题）

> **简历原话** 多路视频流接入：参与多路 RTSP 视频流的拉取与链路调试，每路流由独立线程承载，基于 FFmpeg 完成拉流与拆包并封装为 protobuf 消息统一发布；支持断线自动重连（外层重连、内层读帧的双层循环）与帧间隔统计，便于现场定位链路抖动。
>
> 🔴 **本条两条红线（最容易穿帮）**
>
> 1. **绝不说"解码"**——代码里**没有解码环节**，只说拉流 + 拆包 + 封装。
> 2. **绝不说"我拉的流直接给了流量模块"**——中间隔着解码 + 目标检测跟踪 + 融合，那是算法团队的活。
>
> ⚠️ **背诵优先级**：真正要背的只有三句——① 这模块干嘛的 ② 核心实现（每路一个线程 + 双层循环）③ 我具体参与了什么。

技术点 1：每路流一个线程（1 题）

Q1-1 多路视频流，你整体上是怎么组织的？ [必背]

答： 一个设备管理多路流。内部有一张 **map**，**key 是这路的输出通道名**，**value** 是这路的全部私有状态——视频流地址、连接上下文、它自己的线程、停止标志、连接状态、统计数据。 也就是：**一路流 = 一份私有状态 = 一个独立线程**，彼此之间只有那张 map 是共享的。好处是每路自成一体，加一路减一路只影响表里的一项。

八股补给：

Q1：为什么用智能指针，不用裸指针？

> 三个理由：一是**不用手动 delete**，多线程下不用猜"还有没有人在用"；二是**异常安全**，中间抛异常也能自动释放；三是**所有权写在了类型里**——`shared_ptr` 表示多方共同持有，看代码就知道谁在用。裸指针你得自己记得什么时候能删，多线程下极易出错。

Q2：`shared_ptr` 的引用计数是线程安全的，那还需要加锁吗？

> **需要。** 计数本身的加减是原子的、线程安全，但**它指向的那个对象不是**——多个线程同时读写对象里的成员，照样要加锁。这是个很常见的误解，别以为套了 `shared_ptr` 就万事大吉。

Q3：`shared_ptr` 最大的坑是什么？

> **循环引用**：A 持有 B、B 又持有 A，两边计数都降不到 0，谁也释放不掉，内存泄漏。解法是用 `weak_ptr` 打断这个环——它**不增加计数**，只是"看一眼还在不在"。

追问 1：为什么不让每路流各做一个独立对象，非要一张表？

> 因为框架要能**统一调度**——启动、停止、查状态这些动作是批量下发的。做成一张表，主循环 for 一遍就能让所有路一起动；各做独立对象的话，谁来统一管它们又成了新问题。

追问 2：那张表的"键"为什么是输出通道名，不是流的地址？

> 因为**地址可能会变**（现场改 IP、换相机），而输出通道名是**下游订阅时认的那个名字**，是稳定的契约。用地址当键的话，改一次 IP 下游的订阅关系就全断了。 ⚠️ 地址和通道是**配置里按位置一一对应**的：第一个地址对第一个通道。写错了不会报错，现象是"这路数据跑到了别的通道上"——这是现场最难查的错之一。

追问 3：为什么每路一个线程？用线程池不行吗？

> 不行，因为每路流的**生命周期和阻塞点是独立的**。拉流是长时间阻塞在 I/O 上的，如果塞进线程池，一条流卡住就占掉一个线程，路数一多池子就满了，后面的流根本轮不上。每路一个线程的好处是**谁断了、谁卡了都很清楚**，定位问题时能直接对到某一路。

追问 4：那份私有状态为什么用智能指针装着，不直接放进容器？

> 因为**线程要长期持有它**。直接放对象进去，容器内部调整结构的时候对象地址会变，线程那边抓着的引用就失效了。用引用计数的智能指针可以保证地址稳定，最后一处用完才释放。

追问 5：那停止的时候，线程是怎么退出来的？

> 不是强杀，是**置停止标志 + 让循环自己退出**。线程跑着双层循环，每轮都看一眼标志，看到就跳出、把连接关掉、正常结束。强杀线程会留下没关的连接和没释放的资源。

追问 6：这个每路一线程的模型，最大的缺点是什么？

> **路数一多，线程数就线性涨**。现场十几路还能扛，真到几十路，光线程栈和上下文切换就是一笔开销。要改的话得换成**少量线程 + 非阻塞 I/O** 的模型，但那样代码的复杂度会明显上升——当前规模下不划算，所以维持现状。

技术点 2：视频数据的拉取和拆分（4 题）

Q2-1 你用视频处理库，到底做了哪些事？哪些没做？ [必背] ★★ 最关键的一题

答： 我做的是**拉流和拆包**，**没有做解码**这一步。 拉流就是连上地址、协商参数、把数据接下来；拆包就是把连续的码流切成一个个压缩包。整个模块里**没有任何解码相关的调用**，读到的是**压缩的裸码流**，我原样封装发出去，**谁需要像素自己去解**。 好处是驱动这层职责最轻，也不替下游决定解码策略——下游可能走 GPU 硬解，也可能批量解，那是它的自由。

> 🔴 **红线**：简历写的是"拉流与拆包"，**不是"解码"**。被追问"你是不是做了解码"时直接答：我做的是拉流和拆包，像素级解压交给下游。

> 📌 *自己复习：另有一套 camera/ipcamera 目录的驱动用 CUDA 硬解，**那是另一套，面试别混说。***

八股补给：

Q1：FFmpeg 主要分哪几个库?

> 常用的四个：`avformat`（**容器/协议层**，负责连地址、读写包）、`avcodec`（**编解码层**，真正的解压在这一层）、`avutil`（工具函数和数据结构）、`swscale`（图像缩放和像素格式转换）。**我只用到 `avformat` 这一层**——这正好是"没做解码"的证据。

Q2：怎么判断一个模块到底做没做解码？

> 看它有没有调 **`avcodec_` 开头**的函数。前缀是有规律的：`avformat_` 是容器和流，`avcodec_` 是编解码，`av_packet_` 是包。我们代码里只有 `av_read_frame`（读包）和 `av_packet_unref`（归还），**一个 avcodec 调用都没有**。

Q3：那解码一般放在哪做？

> 下游做，而且边缘设备上基本是 **GPU 硬解**（`avcodec` 配 CUDA 的硬件解码器）。因为 CPU 软解一路 1080p 就很吃力了，多路根本扛不住。这也是我这层不碰解码的原因——**不替下游决定解码策略**，它可能走硬解、也可能批量解、也可能压根不解只存起来。

Q2-3 建立连接那一步，是把整个视频流都拉下来了吗？ [必背] ★★ 最常答错

答： **不是。** 监控相机的视频是**持续不断的实时流**，不像下载文件那样有终点，压根没有"整个"这个概念。 建立连接只做三件小事：**握手建立连接、读一小段数据探测格式、把流的上下文结构体填好**。真正一包一包读数据的是后面那个读帧操作，它在内层循环里被调用，跑一小时就是几十万次。

八股补给：

Q1：RTSP 建连接具体做了什么？

> 大致四步握手：`OPTIONS`（问服务器支持哪些方法）→ `DESCRIBE`（拿 **SDP**，里面写了有几路流、各是什么编码）→ `SETUP`（为每路流建立传输通道，定好用 TCP 还是 UDP）→ `PLAY`（开始推流）。我们说的"探测格式"，主要就是解析 DESCRIBE 拿到的那份 SDP。

Q2：RTSP 和 RTP 是什么关系？

> **RTSP 是控制信令**，负责"开始、暂停、结束"这些命令；**RTP 才是真正传数据的**。类比：RTSP 是遥控器，RTP 是电视信号。所以"建立连接"是 RTSP 的事，"读包"读到的是 RTP 包——两个协议是配合干活的两条通道。

Q3：为什么实时流没有"整个"这个概念？

> 因为服务器那边在**持续编码、持续推流**，没有 `Content-Length`、也没有终点。跟下载文件（有明确长度、读完就结束）完全是两回事。所以代码里只能写成"一直读、读到失败为止"。

Q2-5 那一个包就等于完整的一帧图像吗？ [必背] ★★ 容易答错

答： **不一定**，两个方向都有偏差。 一个是**一帧会被切成好几个包**：比较大的帧，尤其是关键帧，要几个包才能拼成一帧。另一个是**有的包压根不是图像**：参数集、补充信息这些元数据也是以包的形式过来的。 所以简历上写"帧间隔统计"其实是**通俗说法**，严谨讲应该是"相邻两个视频包的间隔"。这个区分值得主动说出来。

八股补给:

Q1：包和帧到底是什么关系?

> 编码后的数据是按 **NALU**（网络抽象层单元）组织的。一帧图像编码后可能被切成**多个 slice**，每个 slice 是一个 NALU；另外 SPS/PPS（参数集）、SEI（补充信息）也各自是独立的 NALU。所以"一个包 = 一帧"这个直觉是错的，两者不在同一个层次上。

Q2：什么是关键帧（I 帧）?

> 就是**能独立解压**的帧，不依赖别的数据，H.264 里 NALU 类型是 **5**。解码器要开始解，必须从 I 帧起步——从中间的 P 帧开始解会花屏。所以抽图、存图、跳转播放都从 I 帧开始找。

Q3：P 帧和 B 帧呢? 丢一个会怎样?

> P 帧**参考前一帧**，B 帧**前后都参考**，所以它们体积小（只存差异）。但丢一个 P 帧，后面参考它的帧也会一起花——**会连带**。丢一个 I 帧更严重，影响的是接下来整整一个 GOP。

Q2-6 读完之后的那个归还操作，是释放这个包吗? [必背] ★★ 容易答错

答： **不是释放包，是减它手里那块数据的引用计数。** 包本身是**栈上的局部变量**，循环里每一轮复用同一个；真正要归还的是它内部指向的那块**数据缓冲区**——库内部有个引用池，这个操作把计数减一，减到零那块内存才真的回到池子。所以初始化那个包只需在进循环前做一次，后面就是反复"读 + 归还"。 为什么重要：一路三十帧跑一小时就是十万次以上，漏一次就泄漏一块数据。我们开发初期踩过内存持续增长的坑，最后就是在**错误分支上漏了归还。**

> 📌 *自己复习：这跟 `free` 完全不同，是引用计数语义。*
>
> **八股补给**:
>
> **Q1：引用计数是什么思路?**

> 同一块数据可能好几个人在用，就记一个数。**谁不用了就减一，减到 0 才真正释放。**所以"归还"只是说"我不用了"，不代表那块内存马上没了——可能还有别人在引用它。

Q2：`av_packet_unref` 和 `av_packet_free` 有什么区别?

> `unref` 是**减引用计数**——包这个结构体还在、可以接着用（我们循环里就是复用同一个）；`free` 是**把包这个结构体本身也释放掉。**我们用的是前者，因为包是**栈上的局部变量**，进循环前初始化一次，之后反复"读 + 归还"。

Q3：为什么这种"借了要还"的设计特别容易泄漏?

> 因为**每一条路径都得还。**正常路径还了、某个错误分支提前 `return` 忘了还，就会漏。一路 30 帧跑一小时是**十万次**级别，漏一次就漏一块。我们开发初期踩过内存持续增长的坑，最后查出来就是某个错误分支上漏了归还。

技术点 3：数据封装与发布（2 题）

Q3-5 往外上报的时候，为什么必须把输出通道名一起传出去？ [必背] ★★ 核心设计点

答： 因为**驱动手里没有发送器，它不知道该往哪儿发。**驱动只知道"我是第几路，我刚拿到了第几个包"；而"第几路该发到哪个通道"是**框架才有的信息**（配置里配的）。 所以回调设计成两个参数：**数据本身，加这路的通道名。**打个比方，数据就是包裹，通道名就是快递单上的收件地址——我打包 + 写地址，快递公司负责投递。**不带地址，快递公司不知道往哪送。** 顺带还有个好处：总表的索引本来就是通道名，将来能直接反查回这路流的状态。

追问：那驱动自己拿着发送器直接发不行吗？为什么非要通过回调绕一圈？

> 不行。驱动是编译成**动态库**的，它根本不知道框架那个发送器在哪、也不知道框架什么时候把它建出来的。更关键的是：一旦驱动直接去依赖发送器，它就**再也离不开框架了**——没法单独编译、也没法单独拿假数据试。回调的作用就是把"我产出数据"和"数据怎么发出去"这两件事切开，驱动只管前者。 而且回调是**构造对象的时候传进来的**，对象一存在回调就必然可用，不会出现"建好了但还没设回调"这种半初始化状态，也就不会出现静默丢数据。

八股补给：

Q1：回调函数到底是个什么东西?

> 说白了就是**把一段函数当参数传给别人，让别人在合适的时候回头来调。** 在 C 里它就是一个**函数指针**——一个变量，存的是函数的地址；在 C++ 里更多是用一个对象把函数"包"起来（比如 `std::function`），这样不光能装普通函数，还能装**成员函数**、装一段当场写的小逻辑。 一句话记：**回调 = 我定"什么时候调"，你定"调了干什么"。**

Q2：那回调和普通的函数调用有什么区别?

> 普通调用是**我去调你**，我写代码的时候就知道要调谁，编译的时候就定死了。 回调是**你来调我**：我写好一段逻辑塞给你，等到某个事件发生了（比如我这边读到一个包），你回头来跑我这段。好处是**你那边不用认识我**——框架不需要知道驱动叫什么、驱动也不需要知道框架怎么发数据，两边靠一个约定好的"函数长什么样"接上就行。

Q3：写一个回调函数要注意什么？最容易踩的坑是什么?

> 三个坑。第一是**生命周期**：回调里用到的那个对象，可能在回调被触发的时候已经被释放了，一调就崩。第二是**别在回调里干重活**——我这个回调是在拉流线程里被调的，里面要是做耗时操作，这路流的帧就直接卡住了。第三是**线程安全**：多路就是多个线程同时调同一个回调，里面碰了共享的东西就得加锁。

Q3-6 那数据最终是发给谁了？发给某个确定的模块吗？ [必背] ★

答： 这一点要诚实：**不是发给某个确定的模块，是发到框架的消息总线上了。** 一路流一个通道，这是**发布-订阅模型**：发布方不需要知道谁在订阅，谁订阅谁收到，**一个订阅者都没有也不报错。**正因为驱动不认识下游，它才必须把通道名交给框架去发；好处是下游增加、删除、换一拨人，驱动一行代码都不用改。 我能看到的订阅方

主要是两个工具：一个把通道里的数据再转推成视频流方便联调，另一个从离线录制包里把视频数据抽出来存文件方便复现问题。**真正的业务下游在公司内部的业务代码里，开源仓库里看不到。** 这句话本身就是既诚实又专业的答法。

追问：那"发到消息总线"和"直接调用下游的函数"，差别到底在哪？

> 差别就在**知不知道对方是谁。**直接调用，我代码里得写死"调某某模块的某某函数"，下游换人、加人、删人，我都得跟着改、跟着重新编译。 发到总线，我只认**通道名**这一个字符串，谁在听我不关心，**一个订阅者都没有也不报错。**所以下游增加、删除、换一拨人，我这边一行代码都不用动。代价是**多一层转发，性能上有一点损耗**，而且出问题时排查链路更长——得一级一级往下查是谁没收到。

八股补给：

Q1：发布-订阅（pub/sub）到底是什么？

> 就是**中间加一个"中转站"。**发的人往某个"频道"里丢数据，收的人提前说好"我订阅这个频道"。两边**互相不认识，甚至不需要同时在线。** 对比一下：直接调用是**一对一、我认识你**；发布订阅是**一对多、我谁都不认识。**

Q2：这种模式和观察者模式是一回事吗？

> 思路是一样的，都是"你变了通知我"，但规模不一样。观察者通常是**同一个进程里**、订阅者直接挂在被观察者身上；发布订阅往往**中间隔着一个消息中间件**，可以跨进程、跨机器。我这边的总线属于后者，所以才需要那个通道名去定位"往哪个频道发"。

Q3：既然方便，为什么不所有地方都用消息总线？

> 三个代价。一是**性能**：多一次拷贝、多一次转发，视频这种高频大数据场景很敏感。二是**调试难**：调用栈断了，看不到"谁调的我"，只能靠日志和通道名去追。三是**顺序和可靠性**：总线一般不保证严格的到达顺序，也不保证一定送达，需要顺序和可靠的地方就得自己再加机制，不能指望它。

> 📌 *自己复习：仓库里另有一套同名前缀的相机驱动，但它用的是另一套消息定义、是独立的 SDK，**不是我们的下游。**同名只是约定了通道名，**同名不等于同一条链路。***

技术点 4：断线重连（1 题）

Q4-1 你说的「双层循环」，具体是怎么配合的？ [必背] ★★ 核心

答： **外层管连接，内层管读数据。** 外层每次进来先试着连一下，连不上就等几秒然后回到外层顶部重新试；连上了就进入内层，内层只要还连着就一直不停地读包。读到一半失败了，就把连接状态置掉、跳出内层，出来之后把这条连接关掉，再回到外层重新开始连。 一句话概括：**外层负责"连不上就重试"，内层负责"连着就一直读"，读断了掉回外层重新连。** 好处是把"连接"和"读数据"这两种完全不同的关注点彻底分开，各自的状态都很简单。

追问 1：外层循环判断的是连接状态吗？

> 不是，它判断的是**循环自己的退出条件**。真正标记"连没连上"的是另一个独立的状态位，由内层在断流时置掉。把这两件事分开，外层就不用关心连接是怎么断的，只看要不要继续重连。

追问 2：连不上就睡几秒再重试，这算轮询吗？会不会占 CPU？

> 不算，这是**带退避的重试**，不是轮询。轮询是转着圈不停检查、CPU 一直耗着；我这个是**真的睡过去**，不占 CPU。而且睡一段时间还能避免设备刚重启时被疯狂连——那是把本来能恢复的设备打得更死。

追问 3：睡眠那句外面还包着一个判断，你知道吗？

> 知道。包的是**停止标志**——睡眠前先看一眼要不要停，要停就立刻退出，不再继续睡。不这么做的话，点了停止还得等这几秒睡完才响应，现场会觉得"点了没反应"。

追问 4：内层出来之后关的是什么？线程也关了吗？

> 关的是**这条连接和它的上下文**，线程**不关**。线程还活着，它会回到外层顶部继续尝试重连——所以这整个双层循环是在同一个线程里跑的，线程只在设备整体停止时才退出。

追问 5：读数据返回失败，就一定代表网络断了吗？

> **不一定**，这是最容易答错的一点。返回失败可能是这一包坏了、可能是暂时没数据、也可能是真的断了。我这里的处理是**一律当作断了跳回外层重连**——因为区分这些的成本很高，而重连的代价很低。这是**有意的取舍**，不是没考虑到。

追问 6：一直连不上会怎样？会不会退出？

> **不会退出，会一直重试下去**，直到设备整体被停止。这是边缘设备的基本素养——现场网络抖动、相机重启都很常见，模块自己先放弃的话，网络恢复后它就再也起不来了。代价是要配合**次数/时长告警**，不然"一直连不上"这件事没人知道。

八股补给：

Q1：跳出内层循环之后，程序到底是怎么回到外层的？

> 靠 `break` 和 `continue` 这两个关键字。 `break` 是**跳出当前这一层循环**，跳出来之后接着往下走（关连接）；外层那个循环没被跳出去，走到末尾自然回到顶部，于是再试一次连接。 所以双层循环不是两个线程、也不是递归，就是**两层 `while` 套在一起**：外层管"要不要再连一次"，内层管"连着就一直读"。

Q2：睡眠的时候线程在干嘛？会不会占着一个核？

> **不会。** 睡眠那一步（比如 `sleep_for` 几秒）是把线程**挂起**，操作系统会把它从运行队列里摘出去，CPU 直接去干别的。等时间到了再把它唤醒塞回队列。 跟它相反的是**忙等**——写一个空转的 `while` 一直判断条件，那样 CPU 会一直跑满。这也是为什么我说这个**不算轮询**：真的睡过去了，就不占 CPU。

Q3：那个停止标志为什么不能就用一个普通 bool？

> 因为它是**两个线程同时读写**的：工作线程在循环里读，主线程点停止的时候写。普通 `bool` 有两个隐患——一是**读到一半的脏值**（虽然 bool 很小、实际上很少见），二是**编译器优化**：编译器看这个变量在当前函数里没被改过，就可能把读取优化成"只读一次放寄存器里"，后面循环再也不去内存里看，于是你那边改成 true 了，这边**永远看不见。** 标准做法是用 **原子类型**（`std::atomic`）来装这种跨线程的标志，它保证读写是原子的、也禁止这种优化。

Q4：循环退出了，线程就结束了吗？主线程要做什么？

> 循环退出只是**线程函数跑完了**，线程对象还在。主线程要调用 `join()` 去等它真正收尾并回收资源；不去 join 也不 `detach()` 的话，程序结束时会直接崩（因为还有线程没被回收）。 简单记：`join` **是"我等你做完"，** `detach` **是"你自己去跑吧我不管了"。** 我们这里是 join，因为要按顺序把连接、上下文都收干净。

Q5：为什么读数据这种操作会"卡住"？卡住了线程不就白等了吗？

> 这就是**阻塞 I/O**。读网络数据的时候，如果对方还没发过来，这个调用默认会**一直等到有数据或者出错才返回**，期间线程是挂起的、不占 CPU。 好处是代码写起来简单；坏处是**一个线程只能盯一条连接**——这正是我们"每路流一个线程"的直接原因。另一种做法是**非阻塞 I/O + 多路复用**，一个线程盯很多条连接，但代码复杂度会明显上升。

条目三：设备与平台运维（5 题）

> **简历原话** 设备与平台运维：读取 Tegra 系统接口采集 GPU 利用率、显存与温度并周期上报，支撑边缘设备健康度监控；独立开发 GUI 批量部署工具（PyQt5），支持多路口配置批量生成与远程下发：Excel 工参经映射规则生成配置，经 SSH/SFTP 并行下发并软重启生效；耗时操作置于线程池、结果经信号槽回传主线程刷新表格，避免界面卡顿；工具已迭代至 V3 并由我独立维护升级。
>
> ⚠️ **本条是两块拼起来的**：**硬件监控**（参与）+ **部署工具**（我独立做的）。部署工具要重点准备——它是唯一能体现"能独立完成一个东西"的部分。
>
> 🔴 **本条两条红线**
>
> 1. **GPU 数据没有落库**：采集完直接上报，别说是"存进数据库再做字段比对"。
> 2. **主动说用了 AI 辅助**（部署工具），别等被问出来。

技术点 1：硬件指标采集（1 题）

Q1-1 你是怎么拿到 GPU 这些硬件指标的? [必背]

答： 没有调任何厂商的私有接口，就是**读文件**。Linux 会把硬件的状态以**文件**的形式暴露在一个特定的系统目录下，所以想知道 GPU 忙不忙、显存占多少、温度多少度，直接打开对应的文件读出来就行，不需要接任何 SDK。

追问 1：为什么读文件就能拿到硬件信息？原理是什么？

> 这是 Linux 的设计哲学——**一切皆文件**。内核把硬件状态以伪文件的形式挂在一个系统目录下，读这个文件本质上是**触发一次内核查询并返回结果**，不是真的在读磁盘。所以开销极小，随便读。

追问 2：一共采哪几个指标？读法一样吗？

> 四个：**利用率、显存占用、显存占用率、温度**，读法各不相同。有的是一次能读出全部字段，有的要按行解析，有的本身就是个整型值。

追问 3：单位换算上有什么坑？

> 有两个地方必须换算，不做的话报出来的数是荒谬的：一是**字节到 MB** 要除两次 1024，少除一次就是一千倍的误差；二是**温度**有的接口给的是毫摄氏度，得再除 1000。这种错误不会崩，只会让监控图看着很离谱。

追问 4：为什么不直接调厂商的 SDK？读文件会不会不稳定？

> 调 SDK 要引入一整套依赖，还得跟着厂商版本走，边缘设备上多一个依赖就多一分风险。读文件的好处是**零依赖、跨版本稳定**——内核暴露的接口不会随便变。稳定性上做过长时间验证，没出问题。

追问 5：采集完之后数据去哪了？落库了吗？

> **没有落库**，采集完直接走上报。这里要说清：我负责的是"采得到、报得出去"，**后面平台怎么存、怎么展示不是我做的。**

追问 6：如果某个指标读不到怎么办？

> 读不到就**跳过这一个、不影响其他指标**，同时打一条日志。不能因为温度读不到就把利用率也丢了——监控最怕的是"整条数据断了"，而不是"少一个字段"。

八股补给：

Q1：读一个文件，底层到底发生了什么？

> 你写的那句"读文件"，最后会走到一次**系统调用**——就是让操作系统内核替你去干。CPU 会从**用户态切到内核态**：你的代码没有权限直接碰硬件，只有内核有。 所以一次读涉及**两次切换**（进去一次、出来一次）加**一次数据拷贝**。这也是为什么高频读小文件不划算——切换本身的开销比读那几个字节还大。 我这个场景是**几秒采一次、读几个小文件**，开销完全可以忽略。

Q2：打开文件之后拿到的那个"句柄"是什么？

> 是**文件描述符**，本质是一个**整数**。内核给每个进程维护一张表，你打开一个文件它就在这张表里占一行，返回的整数就是行号；后面读写都只报这个数字，内核自己去表里查。 关键坑：**这张表是有上限的**，打开之后不关就会一直占着，占满了之后再打开任何文件都会失败——这就是**句柄泄漏**。读系统文件这种"打开、读一行、关掉"的高频动作尤其容易漏。

Q3：读出来的是字符串，怎么变成数字？

> 文件里存的都是**文本**，读出来是字符串，要转数字。C 里用 `atoi`，C++ 里更推荐 `stoi` 这类。 区别是：`atoi` **转换失败会静默返回 0**，你分不清"文件内容是 0"和"内容根本不是数字"；而 `stoi` 会**抛异常**，你能明确知道这行读坏了。 放到我这个场景，区别就是"温度 0 度"和"温度读失败"——这两件事必须能分开，所以宁可用会报错的那个。

Q4：什么叫"一切皆文件"？

> 就是 Linux 把**硬件、进程信息、网络连接**这些东西，都包装成"文件"的样子给你看：打开、读、关掉，操作的接口完全一样。所以你不需要为每种硬件学一套专门的 API，**会读文件就会读硬件状态。** 但"长得像文件"不代表"是文件"：它们**不占磁盘**、读的时候才由内核**现算**出来，所以每次读到的都是实时值。这也是为什么读它们几乎不花时间——没有磁盘 I/O。

技术点 3：部署工具整体设计（2 题）

Q3-1 这个部署工具是干什么的？为什么要做它？ [必背] ★ 最重要的开场题

答： 现场有大量路侧设备要配参数、升版本、查状态。原来的做法是运维拿着一份 Excel **一台一台 SSH 上去敲命令**——慢，还特别容易漏，一个路口配错了可能很久才发现。 所以我做了这款 Windows 桌面工具，把「看

参数表 → 生成配置 → 远程下发 → 看结果」集中到一个界面。一句话说它的价值：**把"一台一台敲"变成"选表 + 点按钮 + 导出结果"。**

追问 1：为什么做成桌面工具，不做成网页？

> 现场同事在办公电脑上用，桌面工具能**直接读本地的 Excel**，也不用额外部署服务器；而且他们拿到的就是一个 exe，**双击就能跑**，不用装环境、不用配浏览器。

追问 2：配置是怎么从 Excel 变成设备上的配置文件的？

> 中间有一层**映射规则**：Excel 的列名 → 配置项的路径。我改的是映射层那几个文件，主流程完全不用动。这样现场说要加两个字段，我只改映射，不用碰生成逻辑。

追问 3：你怎么保证生成出来的配置是对的？

> 两层。一是**生成时校验**：必填项、取值范围、格式先卡一遍，生成不过就不出文件；二是**下发后比对**：把设备上实际在用的配置读回来跟本地这份对比，不一致会把具体差异项打出来——这才是真正能证明"生效了"的手段。

追问 4：是怎么连到那些设备上去的？

> 用 Python 的 SSH 库连上去，文件传输走另一条通道。连接本身做了超时和重试——现场网络环境差，没有超时的话一个连不上的设备能把整批卡死。

追问 5：几十台一起操作，界面会不会卡死？

> 不会。耗时操作都丢进**线程池**，结果通过信号回传主线程刷新表格。界面线程永远只做刷新，不干重活，所以几百台下去界面照样能拖能点。 👉 深挖见**条目三 Q4-1**。

追问 6：这个工具是你独立做的吗？用了 AI 辅助吗？

> 工具是我**独立开发并维护升级**的，已经迭代到 V3。开发过程中**用了 AI 辅助**——主要是搭界面和写重复的胶水代码，业务逻辑和现场适配是我自己做的。 👉 完整版见**条目三 Q3-8。主动说，别等被问出来。**

八股补给：

Q1：连接设备为什么要设超时？不设会怎样？

> 因为默认的连接会**一直卡在那里等**，直到对端响应或者操作系统自己放弃——后者可能是**几分钟**级别。 放到批量场景里，一台设备网络不通，这个任务就**永久占着线程池的一个位置**；来几台不通的设备，池子就被占满，后面所有的任务都排队等着，**整批处理直接堵死**。设了超时，它最多卡那么久就失败返回，位置立刻还回来。 一句话：**超时不只是"怕慢"，更是防止一个坏节点把整体拖死。**

Q2：SSH 和 SFTP 是什么关系？是两个服务吗？

> 不是两个服务，是**同一条连接上的两套用法**。SSH 那套协议里有个"**开子通道**"的能力，SFTP 就是在这条已经认证过的连接上再开一条通道来传文件。 好处是**只认证一次、只占一个端口**（默认 22），防火墙好开，也省去再连一次的开销。所以工具里是"连一次，命令和传文件都走它"。

Q3：为什么要做"下发后把配置读回来比对"这一步?

> 因为**发出去成功不等于生效了**。文件传上去了、命令也返回成功了，但设备那边可能没重启、可能加载的是旧文件、可能配置里有个字段它不认直接忽略了。 只有**把设备上真正在用的那份读回来跟本地对一遍**，才能证明"生效了"，而且不一致时能把**具体差在哪一项**打出来。这比"返回成功就打勾"可靠得多。

Q4：配置文件为什么一般用 JSON / YAML，不用自己定格式?

> 三个原因。一是**不用自己写解析器**，现成的库直接把文本变成程序里能用的结构，省一大堆解析 bug。二是**层级表达能力强**，嵌套的设备、通道这种结构用缩进或者括号写得很清楚，自己定的"每行一个键值对"根本表达不了嵌套。三是**通用**，别的语言、别的工具、甚至人肉看，都能直接读。 代价是**人写起来容易出错**（少个逗号、缩进错一格），所以才要在生成时先**校验**一遍、生成完再**回读比对**一遍。

Q5：这种批量操作，怎么做才不会"做一半"?

> 核心是**让它可重跑**。每一步都设计成"跑一次和跑三次结果一样"（也就是**幂等**），失败了不用回滚，直接再跑一遍就行。 我这个工具里：传文件是覆盖写、创建目录是"没有才建"、校验不过根本不出文件——都是幂等的。真出问题，现场同事重新点一次按钮就好，不需要我教他怎么清理现场。**给现场用的工具，"重跑一次"必须是最安全的解法。**

Q3-8 这个工具是你独立做的吗？用了 AI 辅助吗？ [保底] ★★ 诚实题，必答

答（照实说，这题诚实比包装值钱）： 工具是我**独立开发并持续维护**的，从第一版迭代到第三版。开发过程中我大量使用了 AI 辅助。 说清楚比较准确：**需求、架构、字段映射规则是我定的**，AI 帮我快速产出界面的骨架和一批样板代码，我再**逐行读、改、调**，最后自己打包分发给现场，后续迭代和维护也都是我在做。 第三版主要解决三个现场反馈的问题：域 ID 和订阅主题改成界面可配；证书目录没写权限导致上传失败；把配置的生成和部署合并成一个按钮，避免有人忘记点其中一步。 能佐证的是：它有完整的开发维护记录、有说明文档、有给现场用的操作手册，产物是现场正在用的东西。

追问：AI 写的代码你怎么保证它是对的?

> 三条。第一是**我逐行读过**——AI 出的东西我不直接用，界面骨架和样板代码我会一行一行看，看不懂的地方就查，看懂了才留。第二是**靠现场验收**：工具是给真人在真设备上用的，配置生成错一个字段，设备那边就起不来，所以每次迭代我都在现场环境上跑一遍全流程。第三是**我自己写的那部分才是核心**——Excel 列名到配置项路径的映射规则、连接和重试、权限兜底，这些是业务最关键也最容易错的地方，是我自己写的。 一句话：**AI 提高的是产出速度，判断对错的责任还在我身上。**

八股补给:

Q1：Python 写的程序，怎么变成现场能双击运行的 exe?

> 靠**打包工具**（比如 PyInstaller）。它做的事是：把你的代码、你依赖的第三方库、还有一个**精简版的 Python 解释器**全塞进一个目录或者一个文件里。所以那个 exe 本质是"自带运行环境的压缩包"，运行时会先把自己解开到一个临时目录，再用自带的解释器去跑你的代码。 这也解释了为什么**只拷那个 exe 跑不起来**——依赖目录没跟着走。

Q2：为什么会有"写到了当前目录"这种坑?

> 因为代码里如果写的是**相对路径**（比如直接写个文件名或者 `./logs`），它是相对**当前工作目录**的，不是相对程序所在目录。从桌面双击启动，工作目录可能是桌面；从命令行 `cd` 到别的地方启动，工作目录又是别处——于是日志和产物就散落在不同的地方。 正确做法是**用程序文件自己的位置去算路径**，然后所有读写都基于它。这个坑在 C++ 里一模一样，也属于"部署到别人机器上才会暴露"的典型问题。

Q3：一个要交给别人长期用的工具，除了"能跑"还该有什么?

> 四样。一是**日志**——现场出问题时你不在旁边，只能靠日志复现，而且日志要分级别，别把调试信息也全量写出去。二是**配置外置**：域 ID、订阅主题这些后来就是改成界面可配的，不然现场每换一个场景就得让我重新打包一次。三是**版本号**，出问题能定位"你用的是哪版"。四是**操作手册**——使用者不是你，他不看代码。

> 💡 **说"用了 AI"不减分，说不清自己贡献了什么才减分。** 主动给出"规则我定 / 代码我读我改 / 迭代我维护"这三点，反而说明你能把 AI 用在刀刃上。

技术点 4：并发与界面不卡（1 题）

Q4-1 几十台设备一起操作，界面会不会卡死? [必背] ★★ 高频

答： 会卡——所以**所有耗时的操作一律不放在界面线程**（也就是主线程）。 具体做法是：点按钮之后，主线程里只干两件轻活——读取参数表、把每台设备的连接信息和要传的文件清单组装成一个**任务对象**；然后按一个并发上限**分批**，**每一台设备一个任务**，投进**线程池**里去跑。 后台线程里连上去、传文件、执行命令，干完之后**发一个信号**把结果对象带出去；主线程那边的**槽函数**收到信号，只负责往表格里写一行、打一条日志，**不做任何耗时的事**。 一句话概括：**重活全在后台线程，主线程永远只刷界面。**

追问：那个并发上限是怎么定的？定多少算合适?

> 是**在现场试出来的**，不是算出来的。先定一个小值，拿一批真实设备跑一遍，看总耗时、看有没有大面积超时；再往上加，加到"再加也不快了"就停。 定这个数的目的不是"越快越好"，而是**给下游留余量**——几十台设备同时被连接、同时被传文件，对网络和目标设备都是冲击，冲得太狠会大面积超时，反而比慢慢来更慢。

八股补给:

Q1：Python 有多线程吗？那个 GIL 到底是什么?

> 有线程，但有个叫 **GIL（全局解释器锁）** 的东西：**同一时刻只允许一个线程执行 Python 字节码**。所以 Python 的多线程**不是真正的多核并行**，是"轮流执行"。 它是为了**简化内存管理**才加的——有了它，引用计数就不用到处加锁，单线程程序也更快。代价就是多核利用率上不去。

Q2：既然有 GIL，那我用线程池还有意义吗?

> **有，而且非常大。** 关键要分清任务是 **I/O 密集**还是 **CPU 密集**。 我这个场景是**连 SSH、传文件、等命令返回**——线程绝大部分时间**卡在等网络上**，这时候它会**把 GIL 放掉**给别人用。所以 30 个线程一起等网络，是真的省了 30 倍时间。 反过来，如果是**大量计算**，那线程全程占着 GIL，多线程就白搭，得换**多进程**才能真正用上多核。

Q3：线程池和"来一个任务就起一个线程"有什么区别?

> 两个区别。一是**开销**：起线程要分配栈、要进内核，任务一多光起线程就耗掉不少；线程池是**预先起好一批反复用**。二是**保护**：线程数不受控的话，几百个任务就是几百个线程，系统会直接被拖垮；线程池把并发数**卡在一个上限**，超出的任务排队等着。 这也是为什么我在工具里要**分批投**，而不是一次性把几百个任务全塞进去。

Q4：信号槽为什么是跨线程安全的？它背后是怎么实现的?

> 原理是**事件队列**。后台线程"发信号"这个动作，本质上是往接收方所在线程的一个**队列里塞了一个事件**；接收方线程在自己的循环里取到这个事件，再执行对应的槽函数。 所以槽函数**是在接收方线程里跑的**，从头到尾只有一个线程碰那些界面控件，自然就不用加锁。这也是它比"自己搞个队列 + 一把锁"省事的地方。

Q5：C++ 里没有信号槽，同样的需求怎么做?

> 思路一样：**后台线程算完，把结果塞进一个加锁的队列，主线程去取**。Qt 的信号槽本质就是把这个模式封装好了。自己写的话要自己管三件事：队列要加锁、要有条件变量通知"有东西来了"、还要有一个"退出"的标志让线程能收尾。

技术点 5：远程下发与现场坑（1 题）

Q5-3 你们传文件遇到过权限问题吗？怎么解决的？ [加分] ★★ 最有含金量的一个坑

答： 遇到过。现象是目标目录属主是 root、我用的普通账号没写权限，传输直接报**权限不足**。 一开始我判断的是"目录在不在，不在就创建"，很快就发现不对：目录**明明存在**，就是写不进去。所以我把判据改成了**"当前用户能不能写"——这才是真正的成功判据。 解决套路是分级的：可写就直接过；不可写先试普通用户创建；还不行就走**提权**——用 `sudo` 把目录建出来、再把属主改成当前用户，之后同一账号走 SFTP 就能写了。密码走标准输入，**不拼进命令行**（避免转义，也避免被 `ps` 看到）。 还有个坑：部分设备的 sudo 要求必须有终端，第一次提权会失败，所以失败之后**自动带着终端重试一次**。好处是：现场不用在每台设备上手工配免密 sudo，也不用改成 root 登录。

追问：为什么一开始判断"目录在不在"是错的?

> 因为**"存在"和"能写"是两件事**。目录确实在，但它的属主是 root，我用的普通账号只有读和执行权限、没有写权限——所以判据必须换成**"当前用户到底能不能写"**。 这类 bug 的共同点是：**你检查的是成功的前提，不是成功本身**。后来我养成的习惯是，凡是"检查一遍再动手"的代码，都要问一句——我检查的这个条件，是不是就等于能成功?

八股补给:

Q1：Linux 的文件权限是怎么表示的?

> 每个文件挂着三组权限，每组三个字母：**属主、属组、其他人**，每组的三个字母是 **r（读）、w（写）、x（执行）**。用命令看到的是 `rwxr-xr-x` 这种九个字符。 也可以写成数字：r=4、w=2、x=1，加起来。**755** 就是"属主可读可写可执行，其他人可读可执行但不能写"——所以别人创建的文件，我这个"其他人"是写不进去的，这正是我遇到的那个坑。

Q2：`chmod` 和 `chown` 有什么区别?

> `chmod` 改的是**权限位**（那九个字母），`chown` 改的是**属主**（这个文件归谁）。 我那次的处理是**两个都用**：先用提权把目录**建出来**（因为父目录我没权限写），再 `chown` 把它的属主**改成我这个账号**，之后我走普通连接就能写了。只改权限不改属主也行，但把属主改成自己更彻底，后面所有操作都不用再提权。

Q3：`sudo` 凭什么能让普通用户干 root 的事?

> 因为 sudo 这个程序文件上有个特殊的权限位，叫 **setuid**。带这个位的程序，运行时**不是以启动它的用户身份跑，而是以文件属主的身份跑**——sudo 的属主恰好是 root，所以它跑起来就有 root 的权限。 也正因为这么危险，sudo 才会额外要求**输密码**、要求配置白名单（谁能 sudo、能跑哪些命令），有的环境还要求**必须有终端**——我遇到的就是最后这条。

Q4：为什么在命令行里传密码是不安全的?

> 因为**命令行是公开的**。Linux 上任何一个用户都能通过进程列表看到别人进程的完整命令行参数，密码就明晃晃地挂在上面。 正确做法是**走标准输入**（比如给 sudo 加一个"从 stdin 读密码"的参数），这样密码只存在于进程的内存和输入管道里，不出现在命令行中。顺带还解决转义问题：密码里要是有个空格或者特殊字符，拼进命令行就会被 shell 解释成别的意思。

Q5：把用户填的内容拼进 shell 命令，除了注入还有什么坑?

> 最常见的是**空格把命令截断**。比如文件名里有个空格，拼出来就变成两个参数，命令直接执行错。还有引号、通配符这些特殊字符也会被 shell 先解释一遍。 通用解法是**统一走一个转义函数**，用单引号把参数整个包起来，再处理内部的单引号，这样无论填了什么奇怪的字符串，shell 都只把它当成**一个完整的参数**。

条目四：交通参数检测服务（9 题）

> **简历原话** 4. **检测器框架与数据管道**：负责车头时距、流量检测器的实现，参与抽象基类 + 工厂模式的检测器框架搭建，各类复用同一套「逐帧累积、到点结算」多态流程；并以生产者-消费者模型实现数据链路——上游目标流经 DDS 订阅后写入有界阻塞队列，由独立工作线程消费，数据接收与检测计算解耦。 5. **车辆过线判定**：实现基于停止线几何相交的车辆过线判定，并叠加质量门控（位移阈值 + 横穿方向约束）过滤目标抖动与擦边误判，支撑流量与车头时距统计。
>
> 🔴 **本条三条红线**
>
> 1. **输入是结构化目标流，从没见过像素。**解码、检测跟踪、融合是算法团队的活。
> 2. **通信是 Fast-DDS，不是 CyberRT**（跟设备接入那块是两套中间件）。
> 3. **"上报"不是我做的**：口播讲到"独立工作线程消费、逐帧累积、到点结算"为止。
>
> ⚠️ **三条诚实口径**
>
> 1. 检测器实际是 **6 类**（还有排队 / 车速 / 过街 / 溢流），**没有逆行**；简历收敛写成"负责车头时距、流量检测器"。
> 2. **质量门控只对行人和非机动车严格生效，机动车不拦**（详见 Q3-5）。"支撑流量与车头时距统计"站得住，但**绝不能说成"所有过线目标都过了这道门"**。
> 3. "**负责**…检测器的实现"和"**参与**…框架搭建"是**有意分开**的，别说反、别升格。

技术点 1：检测器框架与多态（3 题）

Q1-1 这个交通参数检测服务，整体上是怎么设计的？ [必背]

答： 它跑在边缘设备上，形态是一个**算法插件**。上游融合感知把结构化的目标流推给我，我订阅收下写进一个**阻塞队列**，由一个工作线程取出来消费——每来一帧就广播给一组检测器各自累加，到配置的周期才统一结算出数。 检测器我用**抽象基类 + 工厂**统一管理，主循环只认基类、不关心具体是哪一类。 一句话：**订阅 → 排队 → 取帧 → 各检测器累加 → 到点结算**，是一条流水线。

追问 1：算法插件是什么意思？它跟宿主程序怎么配合？

> 插件本身就是一个**动态库**，宿主程序启动时按约定的一套 C 接口把它加载起来，我只要把那几个约定的回调实现好就行——初始化、给我数据、取结果。好处是算法和宿主彻底解耦，换算法、升级算法都不用重新编译宿主。

追问 2：你说的"结构化目标流"到底是什么？是图像吗？

> **不是图像，是一帧里所有目标的清单**——每条记录带 id、车型、经纬度、速度、车道号。图像解码、目标检测跟踪、多传感器融合那一整套是算法团队做的，我拿到的是他们的结果，**从头到尾没见过像素。**

追问 3：那你是怎么订阅的？收到的是原始数据还是解析好的？

> 订阅通道是**宿主框架那边声明的**，报文到了会通过回调递给我，我拿到的是一段 **JSON 文本**。我在回调里做的唯一一件事就是把它**反序列化成一帧结构化的目标列表**，然后立刻入队返回——**这一步只做解析，不做任何计算**。

追问 4：这个阻塞队列是你自己写的吗？怎么实现的？

> **不是，是工程里既有的一个组件**，我拿来用，没自己造。它内部是**互斥锁 + 条件变量**，底层是个双端队列，还带**容量上限**。我做的是在它上面搭这条生产-消费链路，轮子本身不是我的。

追问 5：为什么非要"阻塞"？队列空的时候那个线程在干嘛？

> 空的时候就**挂起睡过去**，有新数据入队时被唤醒，一点 CPU 都不占。如果写成轮询，线程就得一直转着圈查"有没有、有没有"——边缘设备上这点算力很金贵，不能这么烧。

追问 6：那为什么不直接在订阅回调里算完？非要多这一跳？

> 因为**回调里做重计算会堵住接收**。感知数据是持续来的，回调线程一旦被计算占住，后面的帧就收不进来，直接丢帧。加队列之后"收"和"算"拆成两段，各自按自己的节奏跑，互不拖累——**这就是简历上说的"数据接收与检测计算解耦"**。

追问 7：为什么只有一个工作线程？不怕算不过来吗？

> 当前的数据量下一个线程够用。而且单消费者有个很大的好处：**检测器内部那些累加账本不用加锁**——同一时刻只有这一个线程在改它们，天然线程安全。要改成多消费者并行，就得给每本账加锁，复杂度和出错概率都会上去。

追问 8：你说的"一组检测器"是多少个？它们之间什么关系？

> 准确说是一个**输出通道**：配置里给每个通道写清楚"多久出一次数"和"这一次要哪几类指标"，通道下面再**按车道、按断面各实例化一个检测器**。所以同一类指标可以同时挂在好几个通道上——比如流量既出秒级的、也出 5 分钟的，**那是两本账，各攒各的，互不干扰**。

追问 9：周期是怎么定的？到点之后算出来的数给谁了？

> 周期写在**配置文件**里，每个通道各配各的。"到没到点"的判定是框架既有的能力，我参与的是让服务支持可配置周期——**别说成"这个周期机制是我从零设计的"**。到点出数之后我调的是**宿主框架的发布回调**，把结果报文交出去就完事；后面发到哪个通道、平台怎么收，**不是我做的那一段**。

> 📍 **这条流水线一定会被拆开问**——简历第 4 条里每个词，对应后面哪一题：
>
> - **"DDS 订阅" / "独立工作线程消费"** → 技术点 2 的 Q2-1、Q2-4
> - **"有界阻塞队列"** → Q2-2（隔开的好处）、Q2-3（满了会丢）
> - **"生产者-消费者模型" / "数据接收与检测计算解耦"** → Q2-1 追问、Q2-4
> - **"抽象基类 + 工厂模式"** → Q1-2、Q1-3
> - **"各类复用同一套...多态流程"** → Q1-4
> - **"逐帧累积、到点结算"** → Q1-5
>
> 背法：Q1-1 是**目录**，后面 22 题是**展开**。面试官从你这句话里随便挑一个词往下问，你都能对上号——**答完 Q1-1 别停，等他挑词。**

八股补给：

Q1：宿主和插件之间为什么用一套 C 接口，不直接用 C++ 类?

> 因为 C++ 的**二进制接口不统一**。同一个类，不同编译器、甚至同一编译器的不同版本，编译出来的符号名、对象内存布局、虚函数表的排布都可能不一样——这叫 **ABI**（二进制接口）不稳定。 宿主和插件往往是分开编译的，一旦两边 ABI 对不上，链接或者运行时就会出莫名其妙的问题。**C 的函数调用约定是稳定的**，所以跨模块的边界基本都用 C 风格的函数来做。

Q2：ABI 和 API 有什么区别?

> **API** 是"源码层面怎么调用"——你看到的头文件、函数名、参数，写错了**编译期**就报错。 **ABI** 是"编译成二进制之后还能不能对上"——符号叫什么、参数怎么传、对象在内存里怎么摆、返回值放哪个寄存器。写错了**编译期不报错**，链接期或者运行时才炸，而且极难查。 一句话：**API 错了编译器告诉你，ABI 错了只能靠现象猜。**

Q3：插件这套接口，为什么通常要带一个"版本号"或者"结构体大小"字段?

> 因为宿主和插件是**分别升级**的。宿主要先知道"你这套接口是哪个版本的"，才知道能调哪些函数、结构体里有哪些字段。 常见做法是：注册的时候插件把**版本号**（或者结构体本身的字节数）报上来，宿主一比对就知道兼容不兼容。不这么做的话，老插件配新宿主就会直接读越界。

Q4：跨模块传递内存有什么坑?

> 最大的坑是**谁分配谁释放必须一致**。在插件里 `new` 出来的东西，最好也由插件提供的释放函数去 `delete`——因为两边的堆、运行时库可能不是同一份，你这边 new 的让他那边 delete，行为是未定义的。 所以跨模块的接口一般会长这样：**插件给一个"创建"和一个"销毁"，成对出现**，宿主只管调，不自己释放。

Q5：那插件里能抛异常吗?

> **不能让它跨出模块边界。** 异常往外抛的时候要做栈展开，而展开需要两边对异常处理的实现保持一致，跨模块很容易出问题。 稳妥做法是**在插件的入口函数里就把异常全部接住**——用 `try...catch(...)` 兜住，转成错误码或者日志返回给宿主。这也是为什么你会看到很多库的对外接口都返回错误码，而不是抛异常。

> 📌 *自己复习：链路走法——宿主回调 → 解析 JSON 成一帧 → `put()` 入队 → 工作线程里 `take()` → 双重 for 广播 `update()` → 判周期 → `calculate_once()` 出数 → 发布回调。队列是 `afl::BlockingQueue`（muduo 风格，作者 Shuo Chen，BSD 协议），容量非 DEBUG 时 60，用到 80% 会打 warning。*

Q1-5 简历里写的「逐帧累积、到点结算」具体是什么意思？ [必背] ★★ 核心

答： 这是消费一帧的**两拍**。第一拍是**逐帧累积**：每来一帧就用一个 for 循环把它广播给所有检测器，各检测器只往自己那份**容器（账本）里累加，这一步不出数。 第二拍是到点结算**：用**帧时间戳**判断有没有跨过配置的周期边界，跨过了才让每个检测器把容器里的数算成结果、并且**算完就清零**，没跨过就直接取下一帧。 分开的好处是统计窗口干净——算完即清零，相邻两个周期不会重叠、也不会重复计数。

追问：那"周期"是从哪来的？

> 是**配置项**，配置里写的周期是多少就按多少聚合。触发逻辑是框架既有的能力，我参与的是让服务支持可配置周期——**别说成"这个周期机制是我从零设计的"**。

八股补给：

Q1：判断"到没到点"，用系统时间还是数据里的时间戳？

> 应该用**数据里带的时间戳**，也就是这一帧是什么时候采集的，而不是"我现在处理到它了"的系统时间。 因为中间隔着**排队**：数据在队列里躺了多久是不确定的。用系统时间的话，队列一积压，本来属于上一周期的帧就被算到了这一周期里，统计出来的数会漂。 用帧自带的时间戳，才算得上是"按真实发生时间聚合"。

Q2：判断跨没跨过周期边界，怎么写才不会漏？

> 最稳的写法不是"上次的时间加上一个周期，看有没有超过"，而是**用时间戳除以周期长度，看商有没有变**（也就是整除/取模的思路）。商变了就说明跨进新周期了。 前一种写法有个隐蔽的坑：如果某一帧的时间戳跳变很大（比如重连后时间跳了一下），用"加上一个周期"去比，可能**一帧跨好几个周期却只结算一次**，中间那几个周期就凭空不见了。

Q3：那些累加的"账本"用什么容器装？为什么？

> 要看**用不用排序**。如果只是"按 id 记住这辆车来过没有"，那就用 `unordered_map`（哈希表那类），查找是常数时间，因为它只关心"在不在"。 如果还要**按时间顺序**处理——比如车头时距要算前后两车谁先谁后——那就得用 `map` 或者 `set`（红黑树那类），它会自动帮你排好序，代价是每次插入要花 **O(log n)** 的时间。 选容器的判据就一条：**要不要顺序**。不要顺序就 `unordered_map`，要顺序就 `map`。

Q4：算完之后为什么要手动清零？不能让它自己重建吗？

> 可以重建，但**重建要重新分配内存**，而清零是**复用同一块**。周期结算是每隔几秒就来一次的，跑几个月就是几十万次，每次都重新分配一次内存，对边缘设备是一笔不必要的开销，还会产生内存碎片。 所以常见做法是**容器留着、只调它的 `clear()`**，容量还在，下次直接接着用。 顺带一提：C++ 里"清空一个容器"和"释放它的内存"是两件事，想把内存也还回去要另外做（比如跟一个空容器交换）。

Q5：累加一堆浮点数最后求平均，有什么要注意的？

> 主要是**精度**。浮点数不是精确的，累加的次数越多，误差越容易积累；而且**先加大的再加小的**和**先加小的再加大的**，结果可能不一样。 我们这个场景是统计车流量、车速，精度要求没那么苛刻，误差可以忽略。但如果是在算钱或者做长时间积分，就得考虑换整数（比如把单位缩小成整数来算）或者用专门的累加算法。

Q1-6 你们一共支持几类检测器？都算什么？ [必背] ★★ 诚实题

答： 代码里是 **6 类**：流量、排队、车速、车头时距、过街行人、还有溢流。简历上我收敛写成"负责车头时距、流量检测器的实现"，是因为这两类是我真正动手做的。 每类的产出不一样：流量出的是分车型的通过数和折算值；排队出的是排队长度和排队车辆数；车速出的是区间平均车速；车头时距出的是前后两车通过同一断面的时间间隔；过街是统计行人过街；溢流是判断路口会不会堵死。 说一下，**没有逆行检测**，别顺着面试官的话头承认。

追问：那几类的差别大吗？能统一到一套接口里？

> 差别主要在"吃进去之后怎么记、吐出来的时候怎么算"，以及**统计的空间粒度不一样**——有的按车道出、有的按整个断面出、有的按路口分支出。但这些差异都封在子类内部，主循环看到的还是那两个接口。

八股补给：

Q1：这 6 类在代码里是怎么区分的？用字符串还是枚举？

> 两处都要有，而且**要能互相转换**。配置里写的是**字符串**（人类可读，现场才好改）；程序里判断的时候用的是**枚举**（一个整数，比较好得快、也不容易写错）。 所以工厂里做的就是一件事：**拿配置里的字符串去查一张表，查到对应的枚举，再按枚举去 new 对应的子类**。这张表通常是个 `map`，key 是字符串、value 是枚举。

Q2：`enum` 和 `enum class` 有什么区别？该用哪个？

> 老式 `enum` 定义的那些名字，会**直接散到外面的作用域里**，两个枚举里都有同名的值就会冲突；而且它能**隐式转成 int**，容易跟整数混着用、写出莫名其妙的比较。 `enum class` 把名字**关在类型里面**（要写 `类型::值`），也**不会自动转成 int**。除了要按位做标志位这种特殊场景，一律用 `enum class`。

Q3：为什么"要算哪些指标"要写在配置里？写死在代码里不行吗？

> 因为**现场需求是会变的**。这个路口现在只想看流量，下个月可能要加排队长度；不同路口要的指标也不一样。写死的话，每加一项就得改代码、重新编译、重新发版本到几十台设备上。 写成配置，现场改一行配置、重启服务就生效。代价是**启动时得校验**：配置写错了（比如写了个不认识的类型）要有明确的报错或者日志，不然现象就是"配了但一直没数据"，很难查。

Q4：6 个子类除了那两个接口，还有没有别的共同点可以往上提？

> 有，但要**克制**。能往上提的只有"所有检测器真的都一样"的东西——比如"本周期结算的是哪一段时间""这条通道叫什么"这种记账用的元信息，提到基类里能少写六遍。 不能往上提的是"某一类特有的算法细节"。一旦为了少写代码把特有逻辑塞进基类，其他五个子类就得被迫继承一堆用不上的东西，以后改一个会影响六个。判据是：**提上去之后，是不是每个子类都用得上。**

技术点 2：生产者-消费者数据管道（3 题）

Q2-1 上游的数据是怎么进到你这个模块的? [必背]

答： 通过 **DDS 订阅**收下来的。上游融合感知把结果发布到一个约定的通道上，我这边订阅它，收到之后先把报文解析成结构化的一帧，然后**塞进一个阻塞队列**就立刻返回。 这里要注意：我们这块用的是 **Fast-DDS，不是 CyberRT**——跟设备接入那套是两套中间件，业务上靠标准化的感知消息衔接，代码上彼此不依赖。

追问：为什么不直接在收到数据的回调里算，非要加个队列?

> 因为**回调里做重计算会堵住接收**。感知数据是持续来的，回调线程一旦被计算占住，后面的帧就收不进来，直接丢帧。加队列之后"收"和"算"拆成两段，各自按自己的节奏跑，互不拖累——**这就是简历上说的"数据接收与检测计算解耦"**。

八股补给：

Q1：DDS 到底是什么？跟 Kafka、RabbitMQ 那种消息队列有什么区别?

> 最大的区别是：**DDS 没有中心服务器**。Kafka 那类要有一个 broker 集群在中间转发，所有数据都得过它一趟；DDS 是**点对点直连**——节点启动后互相发现、直接通信，**没有中间节点，也就没有单点故障**。 代价是：**没有 broker 帮你存历史数据**，晚加入的订阅者收不到它来之前发的那些。这是选型时很实际的取舍。

Q2：DDS 里的 QoS 是什么意思？常用的有哪几个?

> QoS 就是"**这条通信要什么质量**"，在创建订阅的时候配好，中间件按你的要求去保证。常用的几个： **可靠性**——是要"尽力发"（可能丢，但快），还是要"保证送到"（会重传，但有延迟）。我们这种持续采样的数据一般用前者，丢几帧不影响统计。 **历史深度**——给新来的订阅者保留最近几条。 **存活性 / 截止时间**——用来检测对端还在不在、有没有按时发。 这就是为什么实时系统爱用 DDS：**传输质量是可配置的，不是写死的。**

Q3：那为什么不直接开个 socket 自己传?

> 自己写 socket 当然能传，但下面这些事都得自己干：节点怎么发现彼此、断线怎么重连、大包怎么拆、多个订阅者怎么管理、跨语言怎么对齐数据格式。 DDS 把这些都做完了，你只要说"我要订阅哪个主题、要什么质量"。而且它是**标准协议**，不同厂商的实现能互通，算法团队和我们用不同语言也能对上。

Q4：把 JSON 文本解析成程序里的结构，有什么坑?

> 三个。第一是**字段缺失**：上游少发了一个字段，你不能假设它一定在，取值之前要先判存在，不然要么崩要么拿到垃圾值。 第二是**类型对不上**：明明是数字的字段，上游发了个字符串过来，解析就失败。 第三是**性能**：JSON 是文本，解析要逐字符扫描，比二进制格式慢得多。所以我们的做法是**回调里只做解析、立刻入队返回**——把耗时的部分挪到工作线程去，接收这头尽量快。

Q2-3 队列满了会怎样？会丢数据吗？ [必背] ★★ 容易答错

答： **会丢，这是有意的。** 容量满了之后，新来一帧就把**队列里最旧的那一帧挤掉**，给新的腾位置。 这是"有损但稳健"的取舍：边缘设备内存有限，与其让队列无限堆积把模块撑爆，不如丢掉最旧的数据保住稳定性——反正交通参数统计是持续采样，丢一帧老数据的代价远小于模块崩掉。 但有一点要分清：**已经被工作线程取走、正在处理的那一帧，绝不会被丢**，丢的只是还在排队等着的。

追问：那不就保证不了每帧必达了？

> 对，本来就**没承诺每帧必达**。这个场景要的是"持续稳定地产出统计结果"，不是"一帧都不能少"。如果真要求不丢，就得换成能持久化或者阻塞上游的方案，代价和复杂度都不是一个量级。

八股补给：

Q1：这种"满了就挤掉最旧的"队列，底层是什么结构？

> 一般是**环形缓冲区**（也叫循环队列）。做法是**一开始就申请一块固定大小的数组**，用两个下标（一个指向队头、一个指向队尾）在数组里循环着走，走到末尾就绕回开头。 好处是**内存一开始就定死了**，不会随着数据涨；而且入队出队都是改两个下标，非常快。代价就是**容量是写死的**，满了必须按策略处理——我们选的就是"挤掉最旧的"。

Q2：队列满了，除了丢最旧的还有别的选择吗？

> 四种，选哪种取决于业务能接受什么。 ① **阻塞等着**——生产者卡住，等有空位再进（不丢数据，但会拖慢上游）。 ② **丢新的**——新来的直接扔掉，保住老的（适合"旧数据更值钱"的场景）。 ③ **丢最旧的**——我们这个（适合"新数据更值钱"，比如实时统计）。 ④ **扩容**——动态变大（内存会涨，边缘设备上不划算）。 判据就一句：**你是更在乎"不丢"，还是更在乎"拿到的是最新的"。**

Q3：阻塞队列 = 互斥锁 + 条件变量，那条件变量是干嘛的？

> 互斥锁解决的是"**别同时改**"，条件变量解决的是"**等某个条件成立**"。 光有锁的话，队列空的时候消费者只能**反复加锁、看一眼、解锁、再看**——那就是轮询，白烧 CPU。有了条件变量，它可以在条件不成立时**把自己挂起**，等生产者入队后通知它，再被唤醒。

Q4：为什么条件变量必须配 `while` 循环，不能用 `if`？

> 因为存在**虚假唤醒**——线程可能在没有收到任何通知的情况下被唤醒（这是操作系统层面允许的行为，各个平台都可能出现）。 用 `if` 的话，被虚假唤醒后就直接往下走去取数据了，而队列其实还是空的，一取就出错。 用 `while` 的话，醒来第一件事是**再检查一遍条件**，不成立就继续睡。所以标准的写法永远是：**加锁 → while（条件不成立）就等 → 干活 → 解锁。**

Q2-6 算出来的数最后是给谁了？上报是你做的吗？ [必背] ★★ 诚实题

答： 出数之后我调的是**宿主框架给的发布回调**，把结果报文交出去就完事了。 **后面发到哪个通道、平台侧怎么收、怎么展示，不是我做的那一段**，我不清楚细节，也不该硬编。 所以简历上我只写到"独立工作线程消费"为止——我能保证的是**到我这一层的输入和输出是对的**：目标流收得进来、指标算得出来、周期边界干净。

追问：那你怎么验证自己算出来的数是对的？

> 三个办法。一是**离线回放**：把一段真实的目标流存下来反复跑，同一份数据跑出来的结果必须每次都一样。二是**跟现场对**：路口人工数一遍车，跟我出的数对一下，量级和趋势对得上就说明链路是通的。三是**盯边界**：周期切换的那几帧、目标突然消失的情况，重点看有没有重复计数或者漏计。 说实话这块是我觉得做得不够的地方——**没有单元测试**，边界条件主要靠现场跑来验证。

八股补给：

Q1：什么叫序列化？为什么不能直接把内存里的结构体发出去？

> **序列化**就是把内存里的数据结构变成一串**可以传输或者存储的字节**；反过来那一步叫反序列化。 不能直接发结构体的原因是：**内存里的排列方式是给本机看的，不是给传输用的。**里面有对齐填充的空字节、有多字节整数在机器上的存放顺序（大小端）、还可能根本就是个指针（发出去对面拿到的是个地址，毫无意义）。 序列化做的事就是**约定一个双方都认的字节排列规则**，把这些差异全部抹平。

Q2：结构体按字节直接发出去，具体会踩哪些坑？

> 四个。一是**对齐填充**：编译器为了访问快，会在成员之间塞空字节，两边编译器或者编译选项不一样，塞的位置就可能不一样。 二是**字节序**：x86 是小端（低字节在前），有的平台是大端，一个 int 发过去直接反过来读。 三是**指针**：结构体里有个指针的话，发过去的是地址值，对面解引用直接崩。 四是**版本**：加了一个字段，老程序按老布局去读，后面全错位。

Q3：JSON 和 protobuf 怎么选？

> **JSON 是文本，人能直接看懂**，调试抓个包就能读，跨语言也没门槛；代价是**体积大、解析慢**（要逐字符扫）。适合配置、调试接口、和第三方对接。 **protobuf 是二进制**，先写一个 schema 文件、用工具生成代码，然后按字段编号打包。好处是**体积小得多、解析快**，而且**加了字段老程序照样能读**（靠字段编号认，不是靠位置）。代价是**人不能直接看**，调试要靠工具。 判据：**要人读、要通用就 JSON；要快、要省带宽、字段会演进就 protobuf。**

Q4：为什么二进制协议加字段之后，老程序还能读？

> 因为它是**按字段编号认的，不是按位置认的。**每个字段在打包的时候都带着自己的编号，解析的一方遇到不认识的编号就**跳过**继续读下一个，不会错位。 而"直接发结构体"是按位置认的——前面多了一个字段，后面所有字段的位置全变了，老程序就会把 A 字段的值当成 B 字段读出来。这是两种方案最本质的差别。

> 💡 这题答"不是我做的"不减分，硬编细节才减分。边界清楚本身就是加分项。

技术点 3：过线判定与防抖（3 题）

Q3-1 怎么判断一辆车真的过了停止线？[必背] ★★ 核心

答： 记每辆车**上一帧的位置**，本帧到了就把"上一帧位置 → 当前帧位置"连成一条**位移线段**，看这条线段跟**虚拟停止线**相不相交。 相交只是必要条件，还得再过两道闸：位移要够大（不能是原地抖），方向要够"横穿"而不是沿着停止线擦过去，而且这辆车之前没被计过数。三条都满足才算真正过线。 停止线不是物理的线，是配置里给这条检测断面画的**一条虚拟线段**。

追问：为什么要用两帧连成线段？直接看当前帧位置不行吗？

> 不行，**只看单帧位置会漏掉高速车**。30 帧的视频两帧间隔三十几毫秒，快车一帧能挪出去一米多——很可能上一帧还在线这边、下一帧已经跳到线那边，两帧的位置都不在线上，单帧判断就认为它压根没过线。连成线段才能覆盖这种"跨帧跳过去"的情况。

八股补给：

Q1："每辆车上一帧的位置"，在代码里用什么存？

> 一个**键值对容器**，**key 是目标的 id，value 是它上一帧的位置**（再加个时间戳）。每来一帧，先按 id 查到上一帧的位置，连成线段算一遍，算完把这个 id 对应的位置**更新成当前帧**。 选容器就看要不要顺序：**只按 id 查**用 `unordered_map`（哈希表，查找快）；要按时间顺序遍历就用 `map`。我们这里是纯按 id 查，所以是前者。

Q2：判断两条线段相不相交，底层到底怎么算？

> 用**叉积**（也叫叉乘、二维行列式）。叉积的符号能告诉你"这个点在一条有向线的左边还是右边"。 判两条线段相交，就是看：**第一条线段的两个端点，是不是分别落在第二条线段的两侧**；反过来再看一次。两边都"分居两侧"，才是真相交。 好处是**全是乘法和减法，没有除法**，所以又快又不会除零——这也是为什么图形和几何代码里到处都是叉积。

Q3：浮点数为什么不能直接用 `==` 比较？

> 因为**浮点数的表示本身就不精确**。比如 0.1 这种数在二进制里是存不准的，两次不同的计算路径算出来的"同一个值"，末位可能差那么一点点，`==` 就判定为不相等。 标准做法是**比差值**：看两个数的差的绝对值是不是小于一个很小的值（叫 **eps**，比如 1e-6）。小于就认为相等。 几何计算里到处都是浮点，所以**判断"是不是 0""是不是相等"都必须用这个方式**，直接写 `==` 是典型的隐蔽 bug。

Q4：那"参数在 0 到 1 之间"这个判断，是什么意思？

> 这是把线段写成**参数方程**的写法：线段上任何一点都可以表示成"**起点 + t × 方向**"，t 从 0 走到 1 就扫过整条线段。 解出来的 t 如果**在 0 到 1 之间**，说明交点**真的落在这条线段上**；如果 t 是 2 或者 -1，那交点在这条线段**延长出去的地方**——两条无限长的直线是相交的，但这两条线段本身压根没挨着。 所以这个范围检查是必须的，不然会把"延长线相交"误判成"线段相交"。

Q3-2 怎么滤掉"停在停止线附近抖一下就算过线"？ [必背] ★

答： 两道闸。**第一道是位移阈值**：这一帧挪动的距离要超过一个下限（不到半米的不算），原地抖动直接忽略。 **第二道是方向**：位移方向跟停止线法线的夹角要够小，大约 60 度以内才认。那些**沿着停止线方向平移**的——比如车贴着线往前蹭——方向是平行的，不算过线。 第三还有一个记录：每辆车判过一次就记下来，**同一个统计周期内不再重复计。**

追问：那这两个阈值是写死在代码里的吗？能不能配？

> 现在**是写死的常量**，没做成配置项。这个我当时也想过：做成配置的好处是现场能调，坏处是**现场同事并不理解这两个数的含义**，调错了反而更难查。 真要改的话我倾向于**给一个合理默认值、但允许配置覆盖**——既保证不改也能用，又给调试留个口子。这算是这块可以改进的一个点。

八股补给：

Q1：判断方向为什么用余弦，不直接把角度算出来？

> 因为**算角度要调用反三角函数**（acos 那类），它是**很慢的**运算——这段逻辑每一帧、每个目标都要跑一遍，一秒就是几千次，能省就省。 而**余弦在 0 到 180 度之间是单调递减的**：角度越大，余弦越小。所以"夹角小于 60 度"和"余弦大于 0.5"是**完全等价的**，没必要真的算出那个角度。 结论：**比较夹角大小，一律比余弦，不要比角度**——省一次反三角函数。

Q2：余弦是怎么算出来的？点积是什么？

> 两个向量的**点积**（对应分量相乘再相加）等于"**两个向量的长度相乘，再乘上夹角的余弦**"。所以把点积除以两个长度，就得到余弦。 几何意义很直白：**两个向量方向越一致，点积越大**；方向垂直，点积是 0；方向相反，点积是负的。 这也是为什么点积能用来判断"这个位移是不是朝着停止线的法线方向走"。

Q3：C++ 里这种"写死的常量"该怎么定义？

> 三种写法，优先级从高到低： `constexpr` ——编译期就确定，能当数组长度用，也能参与编译期计算，最推荐。 `const` ——运行期常量，只读，够用但不如上面那个。 `#define` ——宏，**不推荐**。它没有类型、不参与作用域、还会在预处理阶段被简单替换掉，容易出各种奇怪的问题。 一句话：**能用 `constexpr` 就用，`const` 兜底，别用宏。**

Q4：那这种"写死的数"是不是就该叫魔法数字、一定要消灭？

> 要分情况。**出现在业务逻辑中间、看不出含义的裸数字**才是坏味道，必须提成常量并给个名字。 但**物理阈值这类**（半米、60 度）提成常量主要是为了**好找、好改**，不是为了消灭它——它本质上就是个经验值，你不可能推导出"为什么正好是半米"。 所以判据是：**这个数字的含义，看代码的人能不能一眼看懂。** 看不懂就必须提常量 + 写注释。

> 📌 *自己复习：简历上把这两道闸写作"质量门控（位移阈值 + 横穿方向约束）"——"横穿方向约束"就是上面第二道。代码常量在 `detectors/stop_line_crossing_helper.cpp`：`MIN_MOVE_DISTANCE = 0.5` m、`MIN_COS_TO_NORMAL = 0.5`（即 cos 60°）。口播只说"约半米""约 60 度"，不报常量名。*

Q3-5 你说的那两道防抖，对所有目标都生效吗？[加分] ★★ 细节加分

答： 这里有个容易忽略的点：**只对行人和非机动车严格生效，机动车其实不拦。**代码里是分目标类型决定要不要走这个门控的。 原因也合理：机动车移动干脆、轨迹稳定，不需要卡这么严；但行人走两步位置就来回抖、还经常沿着停止线方向移动，不卡的话会大量误判。 所以简历第 5 条那句"支撑流量与车头时距统计"，要这么理解：门控是过线判定这一环上的**公共零件**，护住的是这两份统计里**行人 + 非机动车**那部分口径——流量本来就要出分车型明细，行人和非机动车就在明细里；车头时距也是**全类型都算**。 至于**机动车总流量**那一栏，机动车本来就不过这道门，所以它不受门控影响。 一句话：**这道门护的是行人 / 非机动车的过线质量，不是所有过线目标**——绝不能吹成"所有目标都过了质量门控"。

追问：那机动车不拦，会不会把贴着线擦过去的也算进来？

> 有可能，但概率低——机动车跟踪稳、轨迹连续，很少原地抖动。我的理解是这里有意只给行人 / 非机动车加严：**慢速蠕行往前挪的车，两帧位移也可能不足半米**，机动车要是也卡这道门，反而会把真实过线漏掉。而原地抖动和沿路方向平行移动这两类问题，恰恰主要出在行人和非机动车身上。

八股补给：

Q1：按目标类型分情况处理，代码里怎么写？if-else 还是 switch？

> 如果判断的是**一个整数或者枚举**，用 `switch` 更合适——它一眼就能看出"一共就这几种情况"，可读性比一串 `if-else` 好。 如果判断的是**字符串、浮点数，或者是范围**（比如"速度大于多少"），那就只能用 `if-else`。 所以判据是：**分支依据是离散的整数值 → `switch`；否则 → `if-else`。**

Q2：都说 switch 比一串 if-else 快，为什么？

> 因为 `switch` 的分支值**在编译期是已知的、而且是连续的整数**，编译器可以把它编译成一张**跳转表**——用值直接算出一个地址跳过去，**一次比较都不用做。** 一串 `if-else` 只能**从上往下一个一个比**，平均要比一半的次数。 但要注意：这是**编译器的优化，不是语言保证的。**分支值很稀疏（比如只有 1 和 1000 两个）时，编译器可能懒得建表，照样退化成挨个比。所以**可读性才是选 switch 的主要理由，性能只是附带的。**

Q3：`switch` 里忘了写 `break` 会怎样？

> 会**穿透**——执行完这个分支后**继续往下执行下一个分支的代码**，不跳出去。这是 C/C++ 里非常经典的坑。 有些情况会故意利用这个特性（多个值走同一段逻辑），但**必须写注释说明是有意为之**，否则后来的人会以为是 bug 给补上 `break`。 另外 `switch` 一定要有 `default` 分支：枚举以后加了一个新值、但这里忘了加分支，至少 `default` 能兜住，不至于静默走空。

Q4：这种"某一类走特例放行"的写法，有什么隐患？

> 最大的问题是**知识散落在别处。**过线判定这块代码单独看是"所有目标都过门控"，只有看了这里才知道机动车被放行了——**新人最容易在这里踩坑**，或者说错口径。 缓解办法有三个，成本从低到高：**① 在入口处写清楚注释**，说明哪类被放行、为什么；**② 把这个规则抽成一个有名字的函数**，比如"这个类型要不要严格门控"，让调用的地方一眼看懂；**③ 挪到配置里**，代价是配置变复杂。 面试里提这个挺加分——**说明你不光能写对，还知道"能跑"和"好维护"是两回事。**

> 📌 *自己复习：`classify_volume_category()` 先定类型 → `stop_line_crossing_helper.cpp:60` 的 `strict` 只对 `PEDESTRIAN` / `NON_MOTOR` 置真，机动车走 `if(!strict) return true` 直接放行；流量输出 `kVolumeDetailKeys = Volume1..Volume5`（Volume1 行人 / Volume2 非机动车 / Volume3~5 机动车），`Volume` 机动车总流量只加 Volume3~5。*

附录：通用追问（4 题）

Q1 你在这个项目里具体是什么角色？哪些是别人做的？ [必背] ★★ 最重要的一题

答： 我是实习生，主要做了两块：**设备接入与数据链路**，以及**交通参数检测服务**。 具体的：**接口怎么抽象、工厂和自注册怎么设计、多路拉流这一块的实现和调试、GPU 指标的采集，还有交通参数这边的检测器框架、数据管道和车辆过线判定**，这几块是我在 mentor 指导下负责开发和调试的。整体架构是 mentor 定的，我负责把具体模块落地。 部署工具是我独立完成的。中间的解码、目标检测跟踪、多传感器融合那部分是算法团队做的，我不参与——我拿到的是他们的结构化结果。

> 💡 这题的打分点在于**边界清晰**。说"我参与了 X、Y，Z 不是我做的"，比含糊地说"这个项目我都接触过"靠谱得多。

八股补给：

Q1：多人协作写代码，怎么避免互相改坏？

> 靠两样：**分支**和**评审**。 **分支**：每个人在自己的分支上改，改完再合并回主线。这样你写一半的东西不会影响别人，也不会出现"我这边正在改、他那边也在改同一个文件然后互相覆盖"。 **评审**（Code Review）：合并之前必须有人看过你这版改动。它抓的不只是 bug，还有"这段以后别人能不能看懂""这个做法跟现有的一致不一致"。

Q2：评审的时候到底看什么？

> 按优先级：第一**对不对**（边界条件、空指针、资源有没有释放、多线程有没有加锁）；第二**好不好改**（命名、重复代码、有没有把常量写死在逻辑中间）；第三**跟现有的一致不一致**（同一个项目里两套风格，后面的人最痛苦）。 一个很实用的经验：**一次评审的改动别太大**。几百行的改动没人能认真看完，评审就变成走形式了。

Q3：怎么知道某一行代码是谁、什么时候、为什么改的？

> 用版本管理工具的追溯功能（Git 里的 `blame`），它能显示**每一行的最后一次修改是哪个提交、谁提的**。配合提交信息，就能知道当时为什么这么改。 这在实际排查里特别有用：看到一行奇怪的代码，先查它被改的历史，往往注释里没有的原因就出来了。**所以提交信息要写"为什么改"，而不是只写"改了什么"**——后者看代码就知道，前者看不出来。

Q2 你遇到过最难的调试是什么？怎么定位的？ [加分] ★

答（重点是"怎么查"而不是"多难"）： 印象最深的是加载失败那个。现象是某个设备类型在目标机器上就是注册不上，但**在我自己的开发机上是好的**——这种"环境相关的失败"最难查。 我的排查三步：先确认不是代码问题（同一份代码在开发机跑得通）；再缩小到环境差异（是不是依赖库缺失）；最后用查依赖的工具对比两台机器，发现目标机上少了一个间接依赖。解决办法是**把依赖检查前置**——加载之前先确认依赖都在。 教训是：**凡是"在某台机器上不行"的问题，八成是环境的锅，先把环境差异列出来比对，比闷头看代码有效得多。**

八股补给：

Q1：程序崩溃之后怎么查？那个 dump 文件是什么？

> 程序崩的时候，操作系统可以把**崩那一刻整个进程的内存和寄存器状态**存成一个文件，这就是 **core dump**（核心转储）。 有了它，就能用调试器**把它打开、还原出崩的时候的调用栈**——看到是哪一个函数、哪一行、当时的变量是什么值。没有它的话，就只能靠猜。 前提是**要打开这个开关**（系统默认往往是关的，还要放开文件大小限制），这属于"出事前就该准备好"的东西。

Q2：调试器看调用栈，到底在看什么？

> 调用栈就是"**我是怎么一路走到这一行的**"——谁调了谁、谁又调了谁，从最外层一直排到崩的那一句。 有了它，即使是别人的代码崩了，你也能看出**是不是你的参数传错了**。这会比"看着代码想"高效得多。 有个前提：**编译的时候要带上调试符号**（`-g` 那个选项）。发布版本一般会去掉符号，文件小很多，但崩了之后栈里只剩地址、看不出函数名——所以**关键环境要留一份带符号的版本**。

Q3：内存一直涨，怎么确认是不是泄漏？

> 三步。第一步先**确认它真的在涨**：隔一段时间记一次内存占用，画个曲线，看是不是单调上升（程序正常时占用会上下波动，不会一直涨）。 第二步**缩小范围**：把功能一个个关掉，看关到哪个之后不涨了，就定位到那个模块了。 第三步**定位到代码**：用专门的内存检测工具跑一遍，它会告诉你是哪一行分配的内存最后没释放。 我那次是靠前两步定位到"读包那个分支没归还"的——**大多数时候不需要一上来就上工具，先能稳定复现、再二分缩小范围就够了。**

Q3 你的视频流用的是什么协议？为什么选它？ [必背]

答： 用的是 RTSP，实时流传输协议。选它主要有三个原因：第一它**支持双向控制**，除了拉流还可以发暂停、跳转这类控制指令；第二它**延迟比较低**，适合实时监控这种对时效敏感的场景；第三它是**安防监控领域的行业标准**，市面上绝大部分网络相机都原生支持，我们不用为了接某个厂商去做私有适配。 它的工作方式是：先通过协议完成握手和参数协商，然后视频数据才通过另一个通道持续过来，控制和数据是分开的。

追问：那跟 HTTP-FLV 有什么区别？

> HTTP-FLV 基于 HTTP，穿透性更好（走 80 端口基本不会被防火墙拦），但它只能单向拉流、没有控制能力，而且延迟相对高一些。路侧监控看重低延迟和控制能力，而且设备本身原生支持 RTSP，所以选它比较自然。

八股补给：

Q1：RTSP、RTP、RTCP 这三个到底是什么关系？

> 分工不同：**RTSP 管控制**——握手、协商参数、发播放暂停这些指令，它本身**不传视频数据**。真正传数据的是 **RTP**，它是另一条通道。 **RTCP** 是给 RTP 打辅助的，负责汇报"发出去多少包、丢了几个、网络抖不抖"，让你能知道链路质量。 一句话记：**RTSP 是遥控器，RTP 是水管，RTCP 是水表。**

Q2：控制走 TCP，数据走 UDP，为什么分开？

> 因为**要求不一样**。控制指令**一个都不能错、一个都不能丢**，所以用 **TCP**——它保证可靠、保证顺序，代价是有确认和重传，会慢一点。 视频数据**量大、实时性要求高，偶尔丢一包只是花一下**，如果用 TCP，一个包丢了后面的全得等着重传，画面就卡住了。所以用 **UDP**——不管到不到，到了就播。 这就是典型的**按需求选协议**：要可靠就 TCP，要实时就 UDP。

Q3：TCP 和 UDP 最本质的区别是什么？

> 四个。① **要不要先连**：TCP 要三次握手建立连接，UDP 直接发。② **保不保证到**：TCP 丢了会重传，UDP 丢了就丢了。③ **保不保证顺序**：TCP 保证按发送顺序收到，UDP 不保证（后发的可能先到）。④ **有没有流控**：TCP 会根据网络情况自动降速，UDP 不管，有多少发多少。 所以 TCP 像**打电话**（先接通、说清楚、没听清让你重复），UDP 像**喊话**（喊完就完，听见多少算多少）。

Q4：那视频用 UDP 丢了包，画面不会花吗？

> 会，但**可控**。视频是压缩过的，一个包往往不是一个完整画面（可能只是画面的一部分，也就是一个分片），丢一个只是**那一块花一下**，下一帧就补回来了。 而且关键帧（完整的那一帧）会周期性重发，最多卡一下就恢复。真正不能丢的是**关键帧和参数信息**（解码必需的那几个），这些一般会走可靠的通道单独保证。

Q5 你觉得自己这个项目里有哪些做得不够好的地方？ [加分] ★ 必问收尾

答： 坦诚说有几点。第一**可观测性不够**：加载成功与否、注册了哪些设备、每路流当前健康状态，这些都应该在启动时一目了然地打出来，现在是出事了才去翻日志。 第二**统计是全量累计而不是滑动窗口**：像帧间隔这种，跑久了早期数据会一直留着，反映不了"当下"的状态。 第三**缺少主动告警**：现在基本都是"采集了、记录了、等人来看"，理想情况应该是超阈值就主动推送。 第四**测试不足**：当时的验证主要靠实际部署环境和手工构造场景，**没有系统性的单元测试**——有些边界条件如果当时有单测，根本不会拖到现场才发现。

八股补给：

Q1：单元测试到底测什么？为什么大家都说要有？

> 测的是**最小的那一个单元**（一般是一个函数或者一个类），给它一个输入，断言输出是不是你预期的。 最大的价值不是"现在能不能跑通"，而是**以后改代码时有人替你把关**——你改了一处，跑一遍测试就知道有没有把别的弄坏。这在边界条件多的代码上尤其明显：周期正好卡在某一帧、目标突然消失，这些**靠手跑很难稳定复现，但写成测试就是几行的事**。

Q2：C++ 里写单元测试一般用什么？

> 常用的是 **GoogleTest**（gtest）配 **GoogleMock**（gmock）。前者管断言和用例组织，后者用来造"假的对象"——比如你想测一个依赖网络的函数，就用假对象把网络那层替掉，测试就不用真的连网。 配合 CMake 的话，用它的测试模块注册一下，就能一条命令跑完全部用例并出报告。

Q3：什么叫 CI？为什么要有它？

> CI 就是**每次提交代码，自动帮你编译 + 跑测试**的一套流水线。 没有它，"能不能编过、测试过不过"全靠每个人自觉跑一遍——忙起来就有人忘，坏了的代码就悄悄进去了。有了它，**提交之后几分钟就知道结果**，坏了立刻发现，改的成本最低。 对长期项目来说，它的价值不在"高级"，而在**把"应该做但会忘的事"变成自动的**。

Q4：日志为什么要分级别？一般有哪几级？

> 因为**全打太多、不打又查不了。**分了级别就能**按场景开关**：现场正常跑只看警告和错误，出问题了再把调试级打开。 常见的几级，从粗到细：**错误**（出问题了，必须看）、**警告**（不正常但还能跑）、**信息**（关键节点，比如"启动完成""连上了哪台设备"）、**调试**（排查用的细节，正常时不打）。 还有一条容易忽略的：**日志要带上下文**——哪一路流、哪个设备 id、第几帧。光打一句"出错了"，几十路流同时跑的时候根本没法查。

> 💡 主动承认问题 + 给出改进方案 = 有反思能力。比"我觉得挺好的"强一百倍。